UNIVERSITY OF THE PEOPLE

Designing and Evaluating an Integrated Access-Control Framework for Electronic Health Records in a Saudi Government Hospital

A Capstone Project Report of MSIT 5910

Submitted by:

Taghreed Y. Almirabi (C1958065)

For the partial fulfilment of the requirements for the degree of

Master of Science in Information Technology

Supervised by:

Dr. Stella T. Whyte

Department of CS & MSIT

University of the People, Pasadena, CA 91101, United States

AY2027-T1, October 2026

ABSTRACT

This capstone project designed, implemented, and evaluated an integrated access-control framework for electronic health records (EHRs) in a modeled Saudi government hospital. The problem addressed is fragmentation: role permissions, authentication, clinical context, emergency access, and audit review are usually enforced by separate tools, which produces excessive access, unsafe denials, and incomplete evidence. A Python prototype coordinates six-role role-based access control, patient-assignment and duty context, step-up multifactor authentication, controlled break-glass authorization, HMAC-signed audit events, and SQLite persistence behind one application service. Only synthetic identifiers were used; no hospital system was connected and no compliance claim is made. Evaluation combined 25 automated tests at unit, integration, system, and acceptance levels, a 36-case labeled decision benchmark, a documented 300-request local performance benchmark, a five-criterion interface-clarity heuristic, and a structured validation by two healthcare IT experts. One system test exposed a defect in which a whitespace-only user identifier reached a permit path; it was corrected, and all 25 tests passed on the final release v1.3.0-final (commit 0bb3017). Labeled scenarios achieved 100% decision accuracy with no false permits, mean latency was 3.196 ms with a p99 of 4.205 ms, throughput was 312.18 requests per second, peak incremental memory was 0.127 MB, interface clarity scored 4.60/5, and the expert reviewers scored it 4.85/5 and recommended a controlled pilot. The results show coherent, traceable behavior within the stated prototype boundary; they do not establish production security, clinical usability, or hospital-scale performance. A Docker-based deployment plan and a four-category maintenance plan define the next controlled stage.

Keywords: electronic health records, access control, role-based access control, multifactor authentication, break-glass access, audit integrity, system testing, software maintenance

ACKNOWLEDGEMENT

I sincerely thank my instructor, Dr. Stella T. Whyte, for the detailed feedback that shaped this project, particularly the requests to define measurable success criteria, to state the limits of security-critical tests, to document benchmark conditions, to connect each release to its verified commit, and to apply APA 7 referencing consistently. I also thank the Department Chair and the faculty of the Department of Computer Science and MSIT for the program structure that made this capstone possible, and the University of the People for providing accessible graduate education. I also thank the two expert reviewers who evaluated the prototype and gave candid, practical feedback. Finally, I thank my family for their patience and support throughout the proposal, design, implementation, testing, and reporting stages.

UNIVERSITY OF THE PEOPLE

LIST OF TABLES

Table 1. Stakeholders, Concerns, and Expected Value....11
Table 2. Research Questions, Objectives, and Success Criteria....12
Table 3. Comparative Analysis of Existing Access-Control Approaches....18
Table 4. Implemented Role–Permission Matrix....27
Table 5. Integrated Module Inputs and Outputs....28
Table 6. Audit Event Data Model (audit_events Table)....30
Table 7. Functional and Non-Functional Requirements and Acceptance Measures....31
Table 8. Unit 7 System, Acceptance, and Stability Test Cases....37
Table 9. Integrated Evaluation Results....42
Table 10. Expert Reviewer Ratings on the Ten Validation Statements....44
Table 11. Requirements Traceability Matrix....46
Table 12. Objective Status Against Success Criteria....47
Table 13. Proposed Maintenance and Monitoring Schedule....56
Table 14. Post-Deployment Risks and Controls....57

UNIVERSITY OF THE PEOPLE

LIST OF FIGURES

Figure 1. Integrated EHR Access-Control Architecture (Logical Design).....21
Figure 2. Implemented Prototype Architecture and Module Communication.....22
Figure 3. Controlled Break-Glass Workflow.....29
Figure 4. End-to-End Demonstration and Persisted Audit Summary.....36
Figure 5. Initial Failed Blank-Identifier System Test.....38
Figure 6. Corrected Validation and Focused Passing Test.....39
Figure 7. Complete 25-Test Regression Result.....40
Figure 8. Integrated System Evaluation Chart.....43
Figure 9. Verified Unit 7 GitHub Release.....48
Figure 10. Verified Final Release v1.3.0-final.....49
Figure D1. Reviewer 1 Evaluation Form, Page 1 (Information and Review Steps).....69
Figure D2. Reviewer 1 Evaluation Form, Page 2 (Ratings).....70
Figure D3. Reviewer 1 Evaluation Form, Page 3 (Open Feedback and Declaration).....71
Figure D4. Reviewer 2 Evaluation Form, Page 1 (Information and Review Steps).....72
Figure D5. Reviewer 2 Evaluation Form, Page 2 (Ratings).....73
Figure D6. Reviewer 2 Evaluation Form, Page 3 (Open Feedback and Declaration).....74

UNIVERSITY OF THE PEOPLE

SYMBOLS AND ABBREVIATIONS

API	Application Programming Interface
CI/CD	Continuous Integration and Continuous Delivery/Deployment
ECC	Essential Cybersecurity Controls
EHR	Electronic Health Record
FR	Functional Requirement
HIM	Health Information Management
HMAC	Hash-Based Message Authentication Code
HR	Human Resources
IAM	Identity and Access Management
ISO	International Organization for Standardization
IT	Information Technology
MFA	Multifactor Authentication
NCA	National Cybersecurity Authority (Saudi Arabia)
NFR	Non-Functional Requirement
NIST	National Institute of Standards and Technology
PDP	Policy Decision Point
PDPL	Personal Data Protection Law (Saudi Arabia)
PEP	Policy Enforcement Point
RBAC	Role-Based Access Control
RQ	Research Question
SDAIA	Saudi Data and Artificial Intelligence Authority
SIEM	Security Information and Event Management
SOC	Security Operations Center
SQL	Structured Query Language
SSDF	Secure Software Development Framework
SSO	Single Sign-On
TLS	Transport Layer Security

UNIVERSITY OF THE PEOPLE

CONTENTS

ABSTRACT 2

ACKNOWLEDGEMENT 3

LIST OF TABLES 4

LIST OF FIGURES 5

SYMBOLS AND ABBREVIATIONS 6

CHAPTER 1: INTRODUCTION 9

1.1 Background of the Study 9
1.2 Problem Statement 10
1.3 Objectives of the Study 11
1.4 Scope and Limitations 12
1.5 Significance of the Project 13
1.6 Report Organization 13

CHAPTER 2: LITERATURE REVIEW 14

2.1 Overview of Related Work 14
2.2 Comparative Analysis of Existing Approaches 17
2.3 Research Gap Identification 18

CHAPTER 3: SYSTEM DESIGN AND METHODOLOGY 20

3.1 System Architecture 20
3.2 Methodologies and Tools Used 22
3.3 Data Flow and System Components 25
3.4 Design Diagrams 28
3.5 Functional and Non-Functional Requirements 30
3.6 Ethical and Security Considerations 31

CHAPTER 4: RESULTS AND EVALUATION 33

4.1 Implementation Overview 33
4.2 Output and Screenshots 35
4.3 Evaluation Metrics 41
4.4 Deployment and Version-Control Evidence 47

CHAPTER 5: DISCUSSION 50

5.1 Interpretation of Results....50
5.2 Comparison with Literature Findings....51
5.3 Implications and Limitations....53
5.4 Maintenance and Operational Sustainability....55
5.5 Post-Deployment Risks and Mitigation....56
5.6 Overall Interpretation....57

CHAPTER 6: CONCLUSION AND FUTURE WORK....58

6.1 Summary of Main Findings....58
6.2 Implications and Project Contribution....58
6.3 Lessons Learned and Challenges....59
6.4 Future Work....59
6.5 Final Conclusion....60

REFERENCES....62

APPENDICES....66

Appendix A: Test Commands and Evidence Map....66
Appendix B: Configuration and Deployment Summary....66
Appendix C: Repository and Release....67
Appendix D: Expert Reviewer Validation Instrument and Completed Forms....67
Appendix E: Code Excerpts....75

CHAPTER 1: INTRODUCTION

1.1 Background of the Study

Electronic health records (EHRs) make longitudinal clinical information available across departments and can improve continuity, coordination, and decision making. The same concentration of diagnoses, medications, allergies, laboratory results, and personal identifiers creates a high-value target and a serious duty of confidentiality. Access control in healthcare therefore cannot be reduced to keeping outsiders away: it must ensure that legitimate employees receive only the actions and patient context needed for their work, that urgent care is not blocked, and that exceptional access can be explained afterward.

In a large hospital, those decisions depend on several sources: human resources determines whether employment is active, identity and access management (IAM) assigns roles, an authentication service establishes session assurance, the EHR knows the requested action and patient, care-assignment systems provide clinical context, and monitoring systems retain evidence. If these controls are evaluated independently, a person may keep an old department privilege, an administrator may inherit unnecessary clinical access, or an emergency override may be available without review. This capstone addresses that coordination problem through one explainable decision flow.

The Saudi context makes the topic especially relevant. Health information is sensitive personal data whose access should be limited to the personnel and purposes necessary for health services; the Saudi Personal Data Protection Law (PDPL) provides the privacy foundation, while the National Cybersecurity Authority's (NCA) Essential Cybersecurity Controls address identity and access management, event logging, configuration, and resilience (NCA, 2024; Saudi Data and Artificial Intelligence Authority [SDAIA], 2023). This report uses those sources as design guidance and does not present the prototype as certified, compliant, or approved for a government hospital.

1.2 Problem Statement

The project considers a large Saudi government hospital that uses an EHR and employs staff who change wards, shifts, and responsibilities. The study does not inspect a real hospital deployment; it models a representative environment with six roles and synthetic records, which protects confidentiality and keeps the work feasible while still addressing realistic access decisions. The central difficulty is fragmentation. A directory may hold the user's role, an MFA service may verify identity, an EHR may enforce its own permissions, and a separate monitoring tool may receive logs. When these controls are not connected, a transfer may add the new department without removing the old one; a valid role may permit access without checking whether the staff member is on shift or assigned to the patient; uniform MFA can add friction to routine work while failing to protect a high-risk export; emergency access may be too slow or too broad; and separate logs can make it difficult to reconstruct who accessed which record and why. These are design risks, not accusations about a named hospital.

The problem statement is therefore the following. The modeled hospital lacks a unified, testable method for coordinating identity assurance, role permissions, runtime context, emergency exceptions, and audit review. This fragmentation can create excessive access, inappropriate denial, weak accountability, and inconsistent evidence. The project designs and evaluates an integrated access-control framework that makes each EHR decision limited, explainable, and traceable while preserving rapid access during legitimate clinical emergencies. Table 1 summarizes the stakeholders whose concerns shaped the requirements, from patients, whose information the framework protects, to hospital leadership, which needs measurable control coverage.

Table 1

Stakeholders, Concerns, and Expected Value

Stakeholder	Primary concern	Expected value from the framework
Patients	Confidentiality, accurate records, timely treatment	Fewer unnecessary disclosures and a safe emergency pathway
Clinical staff	Fast access, understandable denials, accountability	Permissions that follow duties without repeated manual requests
HIM and privacy teams	Purpose limitation, disclosure control, evidence	Consistent rules and reviewable audit records
IT, IAM, and SOC teams	Lifecycle events, privileged access, alert quality	Defined ownership, structured events, fewer orphaned rights
Hospital leadership	Patient safety, continuity, compliance risk	Measurable control coverage and a phased roadmap

Note. HIM = health information management; IT = information technology; IAM = identity and access management; SOC = security operations center.

1.3 Objectives of the Study

The project is guided by three research questions. RQ1 asks how stable role permissions and changing clinical context can be combined without creating an opaque policy model. RQ2 asks whether a lightweight prototype can enforce normal, high-risk, and emergency decisions while producing complete and tamper-evident audit evidence. RQ3 asks what functional tests, local performance measurements, and structured maintenance planning reveal about the prototype's readiness for a controlled next stage. These questions keep the study focused on an implemented design and proportionate evidence rather than on a broad claim that a hospital environment has been secured.

Five objectives operationalize the questions: O1 maps six hospital roles to the minimum necessary EHR actions; O2 defines authentication, authorization, context, emergency, and audit requirements with testable acceptance measures; O3 implements a rules-based Python prototype that evaluates synthetic requests and persists signed audit evidence; O4 tests normal, denied, high-risk, emergency, and misuse scenarios at unit, integration, system, and acceptance levels and measures local performance; and O5 obtains structured feedback on clarity and practicality through a five-criterion developer heuristic and a structured validation by two healthcare IT experts (Section 4.3.3; Appendix D). Table 2 connects each objective to its research question, its

success criterion, and the chapter in which the evidence is reported, responding directly to the instructor's earlier feedback requesting explicit research questions, measurable objectives, and success criteria.

Table 2

Research Questions, Objectives, and Success Criteria

RQ	Objective	Success criterion	Evidence
RQ1	O1: Six-role least-privilege matrix	All prohibited role–action pairs denied in the automated test set (100%)	Sections 3.3, 4.2, 4.3
RQ1	O2: Testable requirements	Every functional requirement linked to at least one automated test and evidence item (100% traceability)	Sections 3.5, 4.3
RQ2	O3: Executable prototype with signed, persisted audit evidence	One audit event per evaluated request; altered events detected; break-glass decisions persisted	Sections 4.1, 4.2
RQ2	O4: Multi-level testing and local benchmark	≥95% labeled-decision accuracy with zero false permits; all automated tests passing on the released commit; benchmark conditions, hardware, and latency percentiles documented	Sections 4.2, 4.3
RQ3	O5: Structured usability feedback	Heuristic clarity ≥4.0/5; two-reviewer mean ≥4.0/5 with no unresolved critical concern	Sections 4.3, 5.3; Appendix D
RQ3	Deployment and maintenance readiness	Verified release tag linked to tested commit; documented deployment configuration and maintenance plan	Sections 4.4, 5.4, 5.5

1.4 Scope and Limitations

The completed scope includes six healthcare and technical roles, an action-permission baseline, authentication, account-state, and subject checks, patient assignment and duty context, network and device context, step-up MFA, a patient-specific break-glass route, HMAC-protected audit events, SQLite persistence, automated tests, performance measurement, deployment files, Git traceability, and a post-deployment maintenance plan. The logical design additionally specifies a joiner–mover–leaver identity-lifecycle module and SIEM correlation rules; in the prototype these are represented by account-state and duty flags rather than automated HR integration, a difference reported as a limitation in Chapter 5. The proposal originally anticipated

a spreadsheet-based prototype; during Units 4 and 5 the approach changed to an executable Python prototype because code could be unit-tested, integrated with persistent storage, benchmarked, and released under version control. The study excludes live EHR integration, real employee credentials, protected health information, legal certification, production availability, clinical safety validation, penetration testing, and hospital-scale concurrency. The prototype is consequently an evaluated design artifact, not a clinical product.

1.5 Significance of the Project

The project's practical contribution is integration. RBAC, contextual authorization, MFA, emergency access, audit integrity, testing, and maintenance are often discussed as separate safeguards; the prototype shows how one request can move through those concerns in an ordered and reviewable way. For a hospital team, the resulting artifacts—architecture, rules, tests, metrics, risk register, configuration plan, and release history—can support stakeholder discussion before expensive vendor or infrastructure decisions. For academic work, the project demonstrates how security claims can be tied to explicit evidence and limitations.

1.6 Report Organization

Chapter 2 reviews the healthcare, access-control, authentication, testing, and maintenance literature and identifies the research gap. Chapter 3 explains the system design and methodology, including the architecture, decision algorithm, role–permission matrix, requirements, and evaluation method. Chapter 4 reports implementation, testing, and metric results with a requirements traceability matrix. Chapter 5 interprets the results, compares them with the literature, and presents limitations, deployment, maintenance, and risk implications. Chapter 6 summarizes the contribution, lessons learned, and future work. The references follow APA 7 style, and the appendices provide the test evidence map, configuration summary, repository and release details, the reviewer-validation instrument with the completed forms, and code excerpts.

CHAPTER 2: LITERATURE REVIEW

2.1 Overview of Related Work

2.1.1 EHR Confidentiality and the Saudi Context

Electronic health records support continuity of care because clinicians can retrieve diagnoses, allergies, medications, laboratory results, and previous encounters from one longitudinal record. The same concentration of information creates a serious privacy and safety obligation: unauthorized viewing harms dignity and trust, and unauthorized modification can influence treatment. A systematic review of EHR security and privacy found that access control, authentication, and audit were the most frequently studied safeguards, yet many proposals remained conceptual or were evaluated outside clinical settings (Fernández-Alemán et al., 2013). Saudi research reinforces the need for balance: Almaghrabi and Bugis (2022) reviewed 11 studies concerning EHR confidentiality in Saudi healthcare and identified continuing privacy concerns alongside the benefits of digitization, showing that technical safeguards, staff awareness, and organizational practice must be considered together. The present project therefore treats fragmented access governance as a design problem that can be evaluated without exposing a real organization's vulnerabilities.

The PDPL gives the project a local privacy foundation: health information is sensitive personal data, and access and processing must be limited to the smallest number of personnel and to what is necessary for health services (SDAIA, 2023). ECC-2:2024 provides the complementary security perspective, addressing governance, identity and access management, system protection, event logging, and resilience for national entities (NCA, 2024). The framework is described as aligned with these sources rather than certified as compliant, because compliance would require a formal institutional assessment beyond the scope of a student project.

2.1.2 Access-Control Models for Clinical Work

A hospital can apply several access-control models. Discretionary access control lets an owner grant permissions, which is hard to govern for shared clinical records; mandatory access control uses central classifications and is strong but inflexible. RBAC, formalized in the NIST

standard model, assigns permissions to roles such as physician, nurse, pharmacist, laboratory technician, HIM employee, and IT administrator, so that administrators change a role assignment instead of many individual permissions when a person changes position (Ferraiolo et al., 2001).

Static RBAC alone is not sufficient for the proposed setting: a physician's role may permit chart access, yet the request could occur outside an active shift, from an unmanaged device, or without a treatment relationship. Attribute-based access control (ABAC) can express those conditions, but NIST guidance notes that ABAC policies become harder to audit and administer as attribute sets grow (Hu et al., 2014). Cobrado et al. (2024), in a systematic review of EHR access-control research, found RBAC to be the most frequently used approach while noting its difficulty in adapting to changing context, and an analysis of real hospital access logs showed that clinicians frequently needed access outside their static role assignments, which argues for controlled exceptions rather than rigid roles (Røstad & Edsberg, 2006). The project therefore uses context-aware RBAC: the role establishes a stable baseline, and a small set of understandable contextual checks narrows access at the time of the request, which is easier for reviewers to inspect than a large opaque policy set.

2.1.3 Zero Trust, Authentication, and Emergency Access

Zero trust means that network location alone is not accepted as proof of trust. NIST SP 800-207 separates policy decision and policy enforcement functions and recommends explicit authorization based on identity, resource, and context (Rose et al., 2020), and Vukotich (2023) argues that this approach suits healthcare, where shared workstations, mobile staff, and connected devices make perimeter trust unreliable. In the proposed architecture, the policy enforcement point (PEP) intercepts the request, while the policy decision point (PDP) evaluates role and context before the EHR operation is allowed.

Authentication must resist account compromise without creating constant disruption. NIST SP 800-63B-4 describes authenticator assurance levels, session controls, step-up authentication, and phishing-resistant options (Temoshok et al., 2025). Healthcare MFA research shows that factor choice and deployment context matter (Suleski et al., 2023), and a data taxonomy for adaptive MFA proposes using user, environment, device, and use-case data to decide when stronger assurance is needed (Suleski & Ahmed, 2023). A risk-adaptive design is therefore more suitable than repeating the same challenge for every action: a managed ward

workstation during an assigned shift can use SSO, while remote access, privilege changes, unmanaged devices, and bulk export require step-up MFA.

Emergency access is the most sensitive exception. A strict denial can delay care when a patient arrives unconscious, while an unrestricted override can become a route for inappropriate access. Brucker and Petritsch (2009) formalized break-glass as a controlled extension of an access-control model in which exceptional permissions are explicitly granted, logged, and reviewed rather than hidden in administrator overrides, and the designs reviewed by Cobrado et al. (2024) support combining availability with accountability. The project's break-glass mechanism therefore requires an authenticated clinician, a named patient, a structured justification, a short expiry, immediate logging, and retrospective review; it bypasses selected contextual conditions but never identity verification or auditing.

2.1.4 Audit Integrity, Requirements, and Traceability

Audit evidence is only useful if it is complete and trustworthy. NIST's log-management guidance recommends protecting logs from unauthorized modification, retaining them under policy, and reviewing them regularly (Kent & Souppaya, 2006), and the OWASP Top 10 identifies broken access control as a leading application risk, recommending server-side, deny-by-default enforcement with logging of access-control failures (OWASP Foundation, 2025). These recommendations shaped the prototype's ordered-guard design, its HMAC-signed events, and its logging of denials and MFA challenges as well as permits. Requirements also matter: Letaw (2024) distinguishes functional requirements, which state what a system must do, from non-functional requirements that describe qualities such as performance, reliability, and security, and Summers (2020) emphasizes linking requirements to the project objective. These ideas shaped the report's use of requirement identifiers, test scenarios, thresholds, and evidence, because "strong security" is not directly testable, whereas "all prohibited role–action pairs are denied in the test set" is.

2.1.5 System Quality, Testing, Secure Development, and Maintenance

ISO/IEC 25010:2023 defines a product-quality model that includes functional suitability, performance efficiency, compatibility, interaction capability, reliability, security, maintainability, flexibility, and safety (International Organization for Standardization [ISO], 2023); the model discourages treating a single accuracy percentage as complete quality evidence, so functional

tests, persistence checks, latency measurements, usability observations, configuration controls, and maintenance planning should be interpreted together. ISO/IEC/IEEE 29119-1:2022 describes testing as a layered activity in which unit, integration, system, and acceptance levels answer different questions about the product (ISO, 2022b), and system testing specifically evaluates the integrated product rather than individual functions (Martyniuk, 2026); this principle shaped the progression from Unit 5 policy tests to Unit 7 acceptance and stability scenarios. NIST's Secure Software Development Framework (SSDF) recommends integrating secure practices, vulnerability response, and evidence throughout the life cycle rather than adding security only before deployment (Souppaya et al., 2022), and ISO/IEC/IEEE 14764:2022 defines corrective, adaptive, perfective, and preventive maintenance (ISO, 2022a), a classification also summarized in the course reading (Thales, n.d.).

2.2 Comparative Analysis of Existing Approaches

Table 3 compares the main approaches in the literature against the needs of the modeled hospital. None of them alone combines stable roles, clinical context, risk-adaptive authentication, governed emergencies, and tamper-evident evidence in one executable, tested design, which is the combination the prototype implements.

Table 3

Comparative Analysis of Existing Access-Control Approaches

Approach and source	Strengths	Limitations in a hospital	Relevance to this project
RBAC (Ferraiolo et al., 2001; Cobrado et al., 2024)	Administrable; maps to job roles	Static; ignores shift, device, care relationship	Stable permission baseline for six roles
ABAC (Hu et al., 2014)	Expresses rich context	Policies grow complex and hard to audit	Small fixed context set added to RBAC
Break-glass (Brucker & Petritsch, 2009)	Preserves availability with explicit exceptions	Needs justification, expiry, and review	Separate, time-limited, always-audited route
Adaptive MFA (Suleski & Ahmed, 2023; Temoshok et al., 2025)	Matches assurance to risk	Needs identity-provider integration	STEP_UP_MFA decision for high-risk context
Zero trust (Rose et al., 2020; Vukotich, 2023)	Per-request authorization; PDP/PEP separation	Needs device posture and continuous signals	PDP/PEP separation and per-request evaluation
Audit-trail analysis (Røstad & Edsberg, 2006; Kent & Souppaya, 2006)	Reveals real needs; supports accountability	Reactive; logs must be protected	HMAC-signed events for every decision, including denials

2.3 Research Gap Identification

The reviewed sources provide strong pieces of the solution, but they do not by themselves produce a single operational design for the project setting. The Saudi literature establishes the confidentiality concern; the PDPL and ECC establish privacy and security expectations; NIST explains zero-trust, authentication, logging, and secure-development principles; the EHR reviews compare access-control approaches; and the quality, testing, and maintenance standards define how evidence should be produced and sustained. The practical gap is the connection among those elements: how identity status reaches the access decision, when MFA should increase, how a care relationship narrows a role, what happens during an emergency, which audit fields are required, and how the design will be judged successful.

This capstone contributes a coherent blueprint, an executable prototype, and an evaluation method rather than a new cryptographic algorithm. Its value lies in traceability: every identified problem is connected to a module, every functional requirement has an automated test, and every success claim has a threshold. The design responses are direct: the Saudi privacy context is used without claiming a specific breach (Almaghrabi & Bugis, 2022); RBAC is combined with context, step-up MFA, and a governed exception (Cobrado et al., 2024; Røstad & Edsberg, 2006); evaluation and enforcement are separated and every decision is logged (Rose et al., 2020); STEP_UP_MFA is returned for defined high-risk conditions (Temoshok et al., 2025; Suleski & Ahmed, 2023); break-glass is explicit, time-limited, and always audited (Brucker & Petritsch, 2009); events are HMAC-signed for permits, denials, and MFA challenges (Kent & Souppaya, 2006; OWASP Foundation, 2025); and layered tests, synthetic data, versioned releases, and maintenance controls provide life-cycle evidence (ISO, 2022a, 2022b, 2023; Souppaya et al., 2022). That focus makes the design usable as an implementation roadmap and limits overclaiming: a successful synthetic evaluation shows that the rules are internally consistent and understandable; it does not prove that a production hospital is secure or legally compliant.

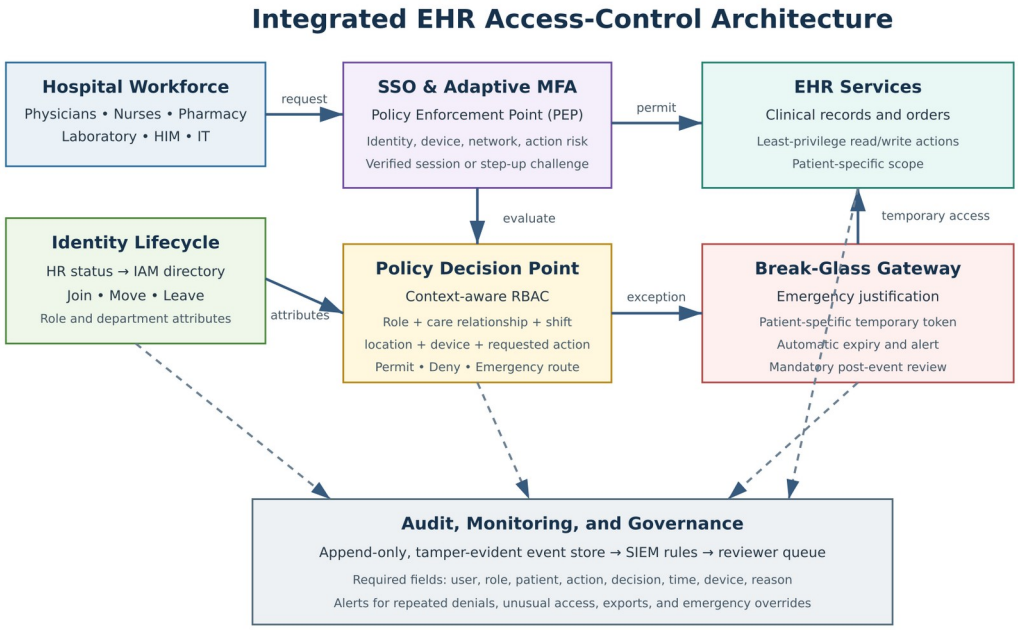
CHAPTER 3: SYSTEM DESIGN AND METHODOLOGY

3.1 System Architecture

Figure 1 presents the logical architecture. The control plane contains identity information, authentication risk rules, the RBAC matrix, contextual policies, and emergency-access rules; the data plane contains the PEP and the EHR services that execute an approved action. Separating these responsibilities keeps policy logic centralized, ensures server-side enforcement, and provides one location for decision evidence. Identity changes originate in human resources and flow to IAM, and every module produces structured evidence for the security information and event management (SIEM) service.

Figure 1

Integrated EHR Access-Control Architecture (Logical Design)

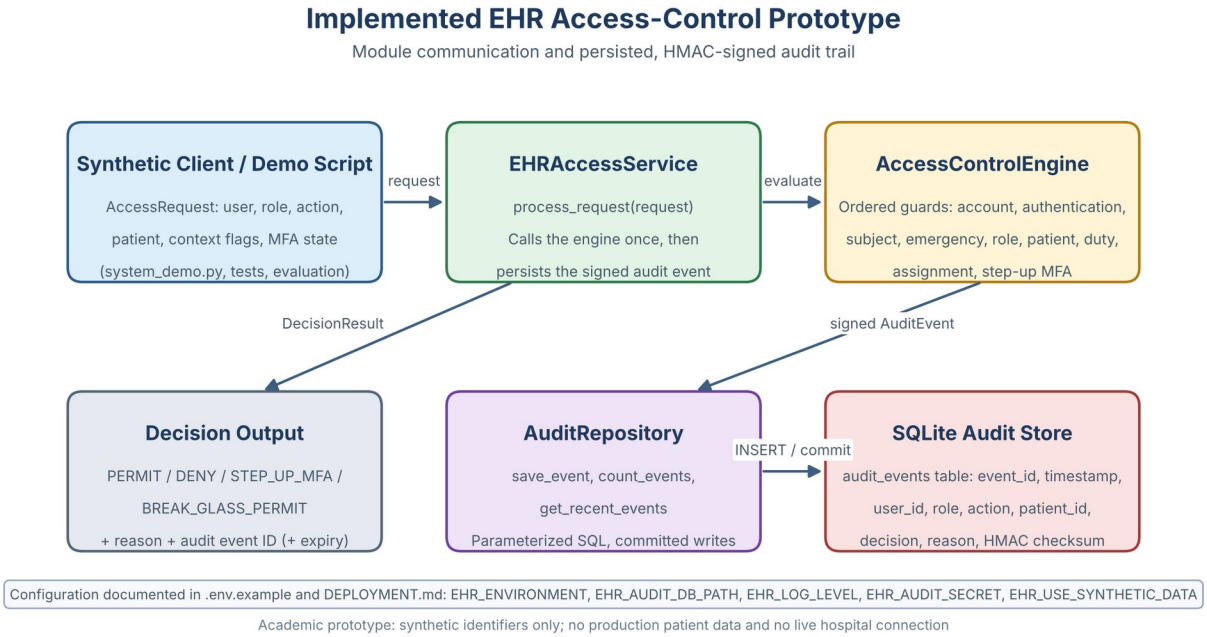


Note. Solid arrows show the authorization path; dashed arrows show security events sent to monitoring and governance. The design connects workforce identity, adaptive authentication, policy decisions, EHR enforcement, emergency access, and governance evidence.

A normal transaction proceeds in five steps: the PEP receives the identity and requested EHR action; the authentication module validates the session and decides whether step-up MFA is required; the PDP retrieves the user's active role from IAM and evaluates the permissions matrix; contextual rules check the care relationship, shift, location, device, patient, and operation; and the PEP forwards a permitted patient-specific operation to the EHR or returns a meaningful denial. Each step writes an event with a shared identifier so that reviewers can reconstruct the decision. Figure 2 shows how the logical design was realized in the prototype: a synthetic client constructs a typed AccessRequest; EHRAccessService calls AccessControlEngine once, retrieves the signed AuditEvent created for that decision, saves it through AuditRepository into SQLite, and returns a DecisionResult. The PEP and PDP are therefore represented by the service boundary and the engine, and the SIEM function by the persisted, queryable audit store.

Figure 2

Implemented Prototype Architecture and Module Communication



Note. The service coordinates authorization and persistence while the engine and repository remain independently testable. Decision labels and configuration names match the released source code.

3.2 Methodologies and Tools Used

3.2.1 Development Approach

The project followed an iterative, unit-by-unit development method planned with a work-breakdown structure and Gantt chart (Oguz, 2025): Units 1 and 2 produced the problem definition, proposal, feasibility checklist, and ethical analysis; Unit 3 the logical architecture, requirements, evaluation plan, and repository; Unit 4 the CI/CD approach; Unit 5 the unit-tested core engine; Unit 6 the integrated service, persistent repository, performance evaluation, and deployment artifacts; and Unit 7 the system and acceptance testing, defect correction, and maintenance plan. Each increment was committed, merged, and tagged, so every result in Chapter 4 can be traced to a repository state.

3.2.2 Core Decision Algorithm

The implemented algorithm uses ordered guards so that a later convenience rule cannot bypass an earlier safety condition. The engine first checks that the account is active and the session is authenticated, then (after the Unit 7 correction) that the user identifier is present and not blank. If the request is flagged as an emergency, the narrowly defined break-glass path is evaluated next, because an emergency request must still satisfy role, patient, and justification conditions but should not be delayed by assignment rules. Otherwise, the engine determines whether the role owns the requested action, requires a patient identifier for patient-level actions, requires on-duty status, requires a clinician–patient assignment for physicians and nurses, and finally evaluates risk. A high-risk action (record export or account management), access from outside the hospital network, or an unmanaged device returns STEP_UP_MFA unless MFA has already been verified. Every path ends in the same method, which creates a reasoned decision and a signed audit event.

```
validate: account active, session authenticated, user identifier
present
if emergency request:
    require clinical role (physician or nurse), patient identifier,
    clinical action, and a justification of at least ten characters
    grant BREAK_GLASS_PERMIT with an expiry timestamp
else:
    require role permission for the requested action
    require patient identifier for patient-level actions
    require on-duty status
    require clinician-patient assignment (physicians and nurses)
    if high-risk action, off-network, or unmanaged device
        and MFA not verified: return STEP_UP_MFA
    otherwise: return PERMIT
create reasoned decision and HMAC-signed audit event; persist; return
```

Step-up MFA is represented as a distinct STEP_UP_MFA decision rather than a denial, so the application can ask the user to reauthenticate and resubmit. This follows NIST guidance that session assurance may be raised when a more sensitive action requires it (Temoshok et al., 2025). The break-glass path returns BREAK_GLASS_PERMIT with an expiry timestamp. The logical design proposed a 30-minute default; the prototype implements a 15-minute constant that a hospital policy would set, and both values are reported so that the difference is visible.

3.2.3 Audit Integrity Mechanism

Each audit event contains an event identifier, timestamp, user, role, action, patient identifier, decision, reason, and checksum. The engine serializes the event fields into a canonical JSON payload with sorted keys and signs it with HMAC-SHA256 using a secret key; verification recomputes the expected value and compares it with hmac.compare_digest, which Python recommends because ordinary equality may expose content-dependent timing behavior (Python Software Foundation, n.d.-a). The mechanism is tamper-evident rather than immutable: it reveals that a stored record no longer matches its signature, but it cannot stop an attacker who holds both the key and database access from replacing the row and its checksum, a limitation that informs the recommendations in Chapter 5.

3.2.4 Tools and Technologies

Python 3.12 provided the implementation language, chosen for readability and its standard-library hmac, json, sqlite3, and tracemalloc modules. pytest executed the test suite because it supports small readable tests, automatic discovery, detailed assertion reports, and reusable fixtures (pytest-dev team, n.d.); SQLite supplied persistent, transactional storage without a separately administered server (Python Software Foundation, n.d.-b); Matplotlib produced the evaluation chart; tracemalloc recorded peak incremental memory (Python Software Foundation, n.d.-c); Docker artifacts documented a portable runtime; and Git with GitHub provided branches, pull requests, tags, and releases. The Unit 4 CI/CD reflection concluded that a lightweight GitHub Actions workflow would run the suite on each pull request, block merges when a security or access-control check fails, and retain test evidence, while automatic deployment to a hospital environment was deliberately excluded (GitHub, n.d.-a).

3.2.5 Evaluation and Testing Method

The evaluation used synthetic request objects and labeled expected decisions, and it combined four instruments. First, the automated test suite verified behavior at unit, integration, system, and acceptance levels; white-box tests were used where rule branches and HMAC behavior were known, and black-box assertions were used where only the request and the expected outcome mattered, with temporary database paths isolating every test. Second, the evaluation script computed decision accuracy as matching decisions divided by labeled

scenarios: the Unit 6 run used a four-case validation set covering an authorized physician, an authorized pharmacist, an unauthenticated nurse, and a justified emergency, and the final run used the 36-case set described in Section 4.3.1, which also counts false permits. Third, the same script measured performance on the integrated service: after ten untimed warm-up requests, 300 sequential physician requests were processed through the engine, the repository, and a temporary SQLite file; in the final run each request was timed individually with time.perf_counter and summarized as the mean, median, 95th-percentile, and 99th-percentile latency, throughput was requests divided by the elapsed time of the timed block, and peak incremental memory was the maximum reported by tracemalloc during that block. Fourth, interface clarity was scored from 1 to 5 on five disclosed criteria—decision labels, reason explanations, input consistency, audit traceability, and ease of learning—and averaged. The acceptance thresholds set before evaluation are those in Table 2: labeled accuracy of at least 95% with zero false permits, 100% denial of prohibited role–action pairs, 100% STEP_UP_MFA escalation of high-risk cases, complete evidence for every emergency case, one signed event per request, all automated tests passing on the released commit, a mean policy decision below the 250-ms design target, 100% requirement-to-test traceability, and a usability score of at least 4.0/5. The design anticipated a 36-case scenario set and external reviewers. The Unit 6 evaluation used a four-case benchmark and a developer heuristic; for the final submission the labeled set was expanded to the planned 36 cases, six per role, covering permitted actions, prohibited role–action pairs, missing context, high-risk escalation, and valid and invalid emergency requests, and two healthcare IT experts completed the structured review in Appendix D. The reviewer validation is practitioner feedback on the prototype's rules and evidence, not clinician usability testing in a live workflow, and it is interpreted within that limit in Chapter 5.

3.3 Data Flow and System Components

3.3.1 Identity and Account-State Handling

The logical design includes an identity-lifecycle module that accepts approved HR events: a join event creates a minimum baseline role, a move event removes rights tied to the old department before adding the new baseline, a leave event disables the account and invalidates sessions, and privileged changes require a separate approver. In the prototype, the outcome of

this module is represented by the account_active, authenticated, and on_duty request attributes, which let the engine deny inactive, unauthenticated, or off-duty subjects without connecting to a real HR system; automated joiner–mover–leaver processing is therefore designed but not yet implemented.

3.3.2 Authentication and Policy Enforcement

The PEP is the entry point for protected requests, and its risk logic is rules-based and explainable: a managed device, the hospital network, and a routine clinical action keep the risk low, while a remote network, an unmanaged device, or a high-risk action raises it. Routine access uses the existing authenticated session; risky actions require step-up MFA, with phishing-resistant authenticators preferred for administrators and remote privileged activity. In the prototype the request carries on_hospital_network, managed_device, and mfa_verified flags, and the engine returns STEP_UP_MFA when a risky request lacks verified MFA.

3.3.3 Context-Aware Authorization and the Role–Permission Matrix

The PDP first checks whether the role is allowed to perform the requested action, then evaluates a deliberately small context set: patient identifier, duty status, and, for physicians and nurses, an active clinician–patient assignment. Missing mandatory attributes result in denial. The rule order matters: a user cannot gain access through favorable context if the role lacks the baseline permission. Each decision includes a reason text so that staff and reviewers can understand the outcome. Table 4 reproduces the permission matrix as implemented in the released source code.

Table 4

Implemented Role–Permission Matrix

Role	Permitted actions	Boundary and context
Physician	view_patient_record; update_clinical_notes; prescribe_medication	Patient identifier and assignment required; eligible for break-glass
Nurse	view_patient_record; update_clinical_notes; administer_medication	Patient identifier and assignment required; eligible for break-glass
Pharmacist	view_patient_record; dispense_medication	Patient identifier required; no clinical-note editing
Laboratory technician	view_patient_record; record_lab_result	Patient identifier required; no unrelated chart actions
Health information manager	view_patient_record; export_patient_record; review_audit_logs	Export is high risk and requires verified MFA
IT systems administrator	manage_user_accounts; review_audit_logs	No clinical-record access by default; account management requires verified MFA

Note. All patient-level actions require a patient identifier; all roles require an active account, an authenticated session, a valid user identifier, and on-duty status. Access from outside the hospital network or from an unmanaged device requires verified MFA for any role.

3.3.4 Emergency Break-Glass Module

Break-glass is an exception to ordinary contextual authorization, not a master account. An authenticated physician or nurse selects the patient, requests a clinical action, and supplies a written justification of at least ten characters; the engine then grants BREAK_GLASS_PERMIT with an expiry timestamp and a signed audit event flagged for review, while non-clinical roles, requests without a patient, non-clinical actions, and empty justifications are denied. The logical workflow (Figure 3) adds the organizational steps around the technical decision: immediate alert, a reviewer who closes or escalates the case, and retained evidence.

3.3.5 Audit, Monitoring, and Evidence

The audit module receives an event for every evaluated request; clinical narrative is excluded, reducing the privacy risk of the logs themselves. Records are written through parameterized SQL inside committed transactions and retrieved in descending timestamp order

with a bounded limit. In production, SIEM rules would flag repeated denials, access without a care relationship, unusual volume, bulk export, privileged configuration, terminated-user attempts, and repeated emergency overrides; an alert is evidence for investigation, not proof of misconduct. Table 5 summarizes the components, their inputs, and their outputs.

Table 5

Integrated Module Inputs and Outputs

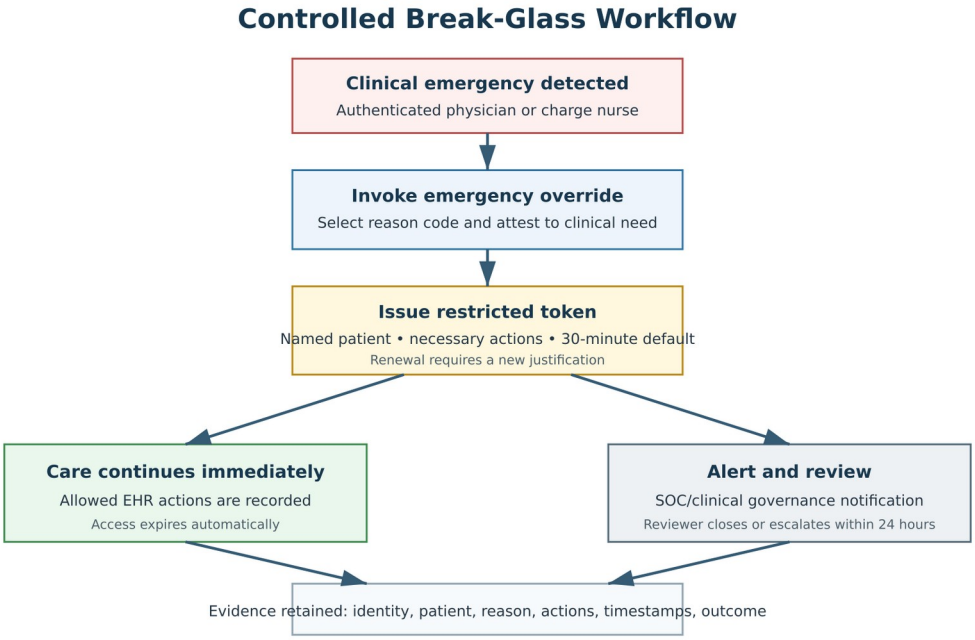
Component	Primary responsibility	Input	Output
AccessControlEngine	RBAC, context, step-up MFA, break-glass, audit signing	AccessRequest	DecisionResult and AuditEvent
EHRAccessService	Coordinate authorization and persistence	Request and module references	Saved decision and retrieval methods
AuditRepository	Create schema; store and query events	AuditEvent or query limit	Persistent row, count, recent events
system_demo.py	Execute end-to-end synthetic workflows	Scenario requests	Console decisions and audit summary
performance_evaluation.py	Measure correctness, speed, memory, clarity	Labeled workload	CSV results and PNG chart
Docker and configuration files	Package runtime and document settings	Source, dependencies, environment	Reproducible deployment baseline

3.4 Design Diagrams

Figure 3 presents the controlled break-glass workflow, which is the most sensitive exception in the design. Table 6 documents the persisted audit-event entity, which is the data model shared by the engine, the repository, and the evaluation scripts.

Figure 3

Controlled Break-Glass Workflow



Note. Emergency access is patient-specific, time-limited, immediately logged, and subject to retrospective review. The 30-minute default shown is the logical design value; the prototype constant is 15 minutes.

Table 6

Audit Event Data Model (audit_events Table)

Field	Type	Purpose
event_id	TEXT, primary key	Random 16-hex-character identifier returned to the caller
timestamp	TEXT	UTC ISO-8601 time of the decision
user_id	TEXT	Synthetic subject identifier (validated non-blank)
role	TEXT	One of the six implemented roles
action	TEXT	Requested EHR action
patient_id	TEXT	Synthetic patient identifier or None
decision	TEXT	PERMIT, DENY, STEP_UP_MFA, or BREAK_GLASS_PERMIT
reason	TEXT	Human-readable explanation of the outcome
checksum	TEXT	HMAC-SHA256 over the canonical payload

3.5 Functional and Non-Functional Requirements

Table 7 lists the requirements that guided implementation and the acceptance measure for each. They were refined from the Unit 3 catalog to match the implemented scope: the former lifecycle requirement (automated transfer and termination handling) was replaced by FR-02, which the prototype can verify, and its automation is retained as future work.

Table 7

Functional and Non-Functional Requirements and Acceptance Measures

ID	Requirement	Acceptance measure and design influence
FR-01	Enforce six-role least privilege	All prohibited role–action pairs are denied; central permission matrix and prohibited-action tests
FR-02	Reject inactive, unauthenticated, or unidentified subjects	Each case returns DENY with a reason and an audit event; ordered validation guards
FR-03	Apply step-up MFA to high-risk context	Export, account management, off-network, and unmanaged-device requests return STEP_UP_MFA without verified MFA
FR-04	Evaluate patient and operational context	Missing patient identifier, off-duty status, or missing assignment is denied
FR-05	Provide governed emergency access	Break-glass requires clinical role, patient, clinical action, and justification; returns expiry; always audited
FR-06	Create complete, tamper-evident decision evidence	One HMAC-signed event per request persisted in SQLite; altered events fail verification
NFR-01	Performance efficiency	Local mean latency and throughput baseline below the 250-ms policy-decision design target; conditions documented
NFR-02	Reliability and recoverability	Regression suite passes on the released commit; backup, rollback, and restore activities in the maintenance plan
NFR-03	Security and maintainability	Separation of duties, version control, configuration plan, pinned dependencies, non-root container

Note. FR = functional requirement; NFR = non-functional requirement.

3.6 Ethical and Security Considerations

Privacy began with the project method. The prototype used fictional staff, patients, departments, and timestamps; no production screenshots, credentials, IP addresses, or patient details were collected. The reviewer validation was voluntary; the form (Appendix D) explained the purpose and the information collected, and each reviewer signed a declaration that the evaluation reflected independent judgement and that no real patient data had been shown. Fairness is also relevant even though the engine is rule-based: a policy may burden night-shift, temporary, mobile, or cross-cover staff more than others, so the test set covers all six roles, off-

duty access, missing assignment, and emergency cross-cover; protected personal characteristics are not authorization inputs; and denials include a reason, with a route for human review in the logical design. Security protections in the design include encryption in transit and at rest, least privilege for service accounts, secret management, session expiry, MFA for sensitive actions, segregation of duties, protected audit retention, and fail-safe behavior: if the PDP is unavailable, normal requests fail closed while the hospital's approved downtime and emergency procedures preserve clinical continuity. Any future deployment would require threat modeling, security testing, privacy impact assessment, change management, and governance approval.

CHAPTER 4: RESULTS AND EVALUATION

4.1 Implementation Overview

This chapter presents the implementation, integration, testing, and evaluation results of the capstone prototype. Units 4 and 5 established the implementation workflow and core authorization engine; Unit 6 joined that engine to an application service, a persistent audit repository, evaluation code, and deployment artifacts; and Unit 7 added system and acceptance tests, a defect correction, and the maintenance plan. The prototype remained within the approved scope: it uses synthetic users and patient identifiers, does not retrieve clinical content, and does not connect to a government hospital network. A functional prototype can show that components communicate correctly and that policy outcomes are reproducible, but it cannot by itself establish production readiness, legal compliance, or clinical safety, so the findings are reported as evidence about the implemented prototype rather than about a deployed hospital service.

4.1.1 Implementation Environment and System Baseline

The implementation uses Python 3.12 and a modular repository structure: source files under src, tests under tests, evaluation outputs under results, deployment instructions under deployment, and documentation under docs. The verified Unit 7 run used Windows 11, Python 3.12.15, pytest 9.1.1, and Matplotlib 3.11.2 in one Python process; requirements.txt at that point pinned pytest 9.0.1 and Matplotlib 3.11.2, and aligning the pytest pin with the verified version was recorded as a maintenance task that the final release completed. On October 17, 2026, the main-branch merge commit d2ac7cd (d2ac7cdd4ae9c99217a074affd515bb8a63298a9) was checked out and the complete suite rerun; all 25 tests passed. The published v1.2.0-unit7 tag resolves to d2ac7cd, so the test evidence and the released source refer to the same verified state. For the final submission the suite was rerun on October 24, 2026, on commit 0bb3017 with the aligned pin, again with 25 of 25 tests passing in 0.41 seconds (Section 4.4).

4.1.2 Core Algorithm Implementation Results

The core engine evaluates a request in the ordered stages described in Section 3.2.2: account state, authentication, and subject identifier first; the emergency path when the emergency flag is set; otherwise role permission, patient identifier, duty status, clinician–patient assignment, and finally the risk conditions that may require step-up MFA. The order makes deny conditions explicit, supports least privilege, prevents an emergency flag from bypassing identity requirements, and provides a reason that can be inspected during review. The six roles and their permissions (Table 4) correspond to job responsibilities rather than broad system membership, and patient assignment, duty status, emergency status, network, device, and authentication context narrow or strengthen the decision, addressing the limitations of static role membership identified in healthcare access-control research (Cobrado et al., 2024). STEP_UP_MFA is returned as a distinct decision so that the application can ask the user to reauthenticate and resubmit (Temoshok et al., 2025), and BREAK_GLASS_PERMIT carries a 15-minute expiration and a signed audit event, deliberately avoiding a silent administrator override. A dedicated test confirms that changing any signed audit field causes verification to fail; the HMAC key in the released code is a synthetic placeholder that production deployment must replace with a managed secret (Chapter 5).

4.1.3 System Integration and Module Communication

The integration architecture uses EHRAccessService as the application boundary. The service receives a request, calls the engine once, locates the audit event created for that decision, saves it through the repository, and returns the decision. Without this small orchestration layer, the demonstration script would need to know how policies are evaluated and how database connections are opened; with it, the engine can be replaced independently and the repository can later move from SQLite to a managed relational database without changing the caller's workflow. The integrated path is therefore: the client constructs a typed AccessRequest; the service invokes engine.evaluate(request), which returns a DecisionResult with the outcome, explanation, audit identifier, and any break-glass expiration; the service calls repository.save_event with the signed event; the repository executes a parameterized INSERT within a managed SQLite connection and commits; and the decision returns to the caller (Appendix E). This gives one audit record for each evaluated request, including denials and

MFA challenges. SQLite was selected because it provides persistent, transactional storage without a separately administered server (Python Software Foundation, n.d.-b), and the database stores security metadata only; the prototype never stores diagnoses, medications, names, national identifiers, or other real patient content.

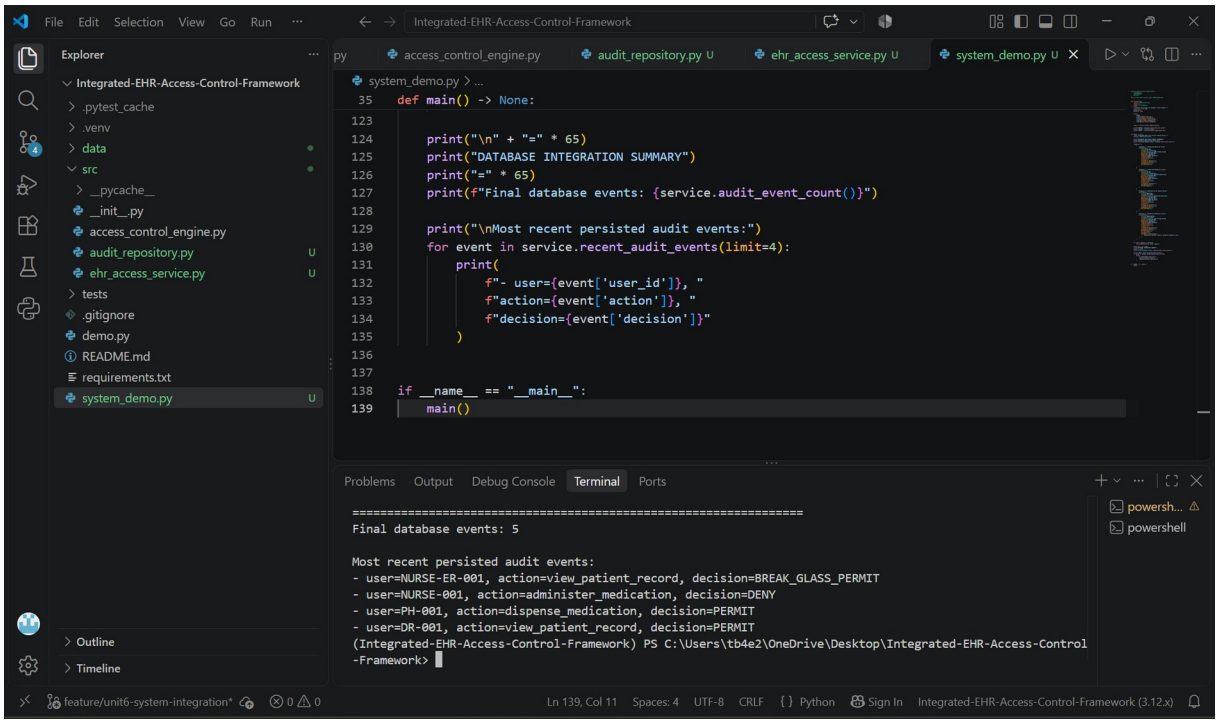
4.2 Output and Screenshots

4.2.1 End-to-End Demonstration

The demonstration program supplies human-readable end-to-end evidence. It processes four scenarios: an assigned physician viewing a patient record, a pharmacist dispensing medication, an unauthenticated nurse attempting to administer medication, and an emergency-room nurse requesting break-glass access with a written justification. The observed output (Figure 4) included PERMIT for the two authorized actions, DENY for the unauthenticated request, and BREAK_GLASS_PERMIT for the justified emergency. The database reported five events because the persistent demonstration file already contained one event from an earlier execution; the four newest rows correspond to the four scenarios. Because retrieval occurred after the decisions were saved, the output validates communication among the scenario driver, service, engine, repository, and database rather than one function in isolation.

Figure 4

End-to-End Demonstration and Persisted Audit Summary



Note. The console displays the persisted event count and the four newest decisions (BREAK_GLASS_PERMIT, DENY, PERMIT, PERMIT) retrieved from SQLite after the scenarios ran.

4.2.2 Testing Levels and Procedures

Testing combined 15 unit tests, three integration tests, and seven Unit 7 system and acceptance tests. Unit tests examine policy behavior in isolation; integration tests instantiate a temporary repository and the real service, submit permitted, denied, and break-glass requests, and inspect the saved rows; and the system and acceptance tests exercise complete workflows and a sequential stability scenario. Temporary paths isolate each database, and pytest's discovery conventions allow the complete suite to run with one command (pytest-dev team, n.d.). The focused Unit 7 file was executed first with the verbose flag, and the complete suite was then executed as a regression check; the verbose log is preserved in results/unit7_full_test_log.txt (Appendix A).

One especially important integration test created a nurse request with emergency=True, assigned_to_patient=False, and a written justification, and asserted that the response was

BREAK_GLASS_PERMIT, that the repository contained one event, and that the stored user and decision matched the response. This guards against a subtle defect in which the engine produces the correct emergency result while the database saves nothing or an inconsistent decision. Table 8 summarizes the seven Unit 7 cases.

Table 8

Unit 7 System, Acceptance, and Stability Test Cases

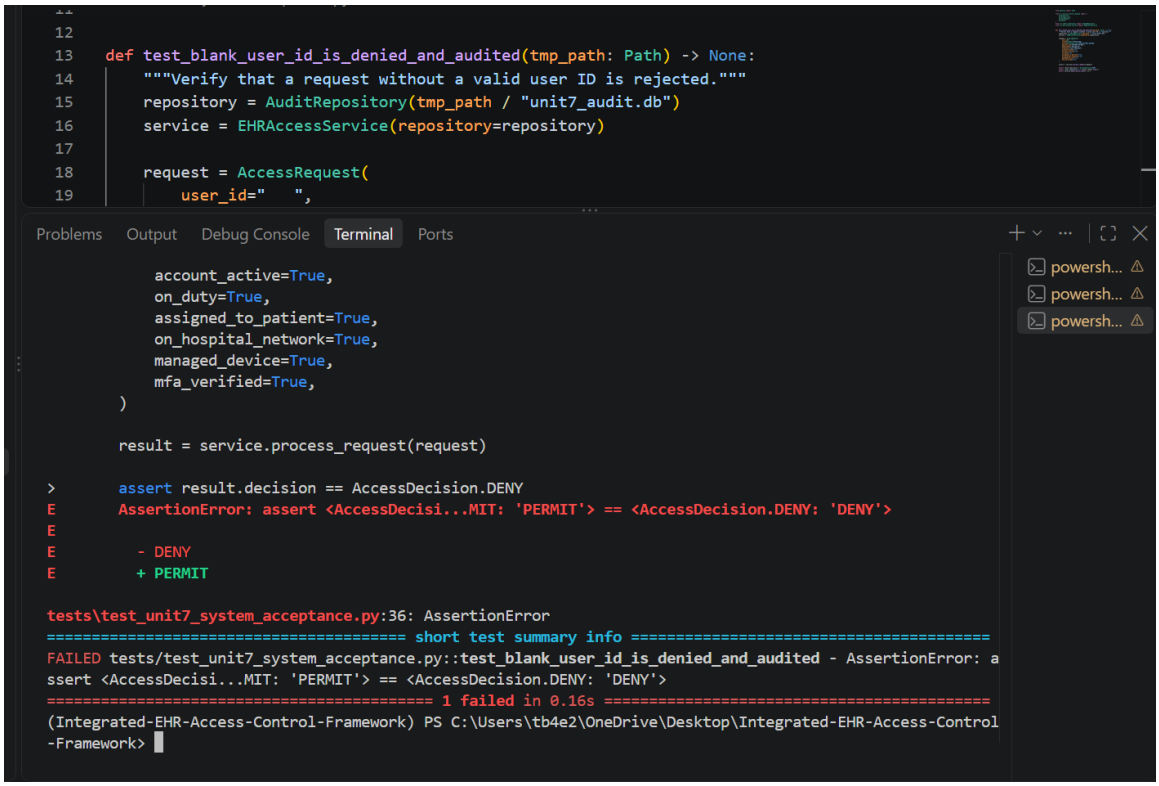
Case and level	Expected behavior	Observed result
Blank subject (system)	DENY; reason mentions the user identifier; one audit event	Initially failed; passed after the validation guard
Assigned physician (acceptance)	PERMIT for the assigned patient	Passed
High-risk export (acceptance)	STEP_UP_MFA without verified step-up authentication	Passed
Break-glass (acceptance)	Time-limited permit plus a persistent audit event	Passed
IT administrator (acceptance)	DENY for default clinical-record access	Passed
Mixed workflow (system)	One complete signed audit event per request	Passed
Fifty sequential requests (stability)	All requests processed; no event loss	Passed

4.2.3 Defect Discovery and Resolution

The first execution of the new system test produced one useful failure. A request whose user_id contained only spaces was permitted because the engine verified account state and authentication but did not validate the identifier before reaching the role rule; the failed assertion expected DENY but received PERMIT (Figure 5). This was security-relevant: an incomplete subject identifier should never enter an authorized audit trail, because attribution depends on a valid subject.

Figure 5

Initial Failed Blank-Identifier System Test



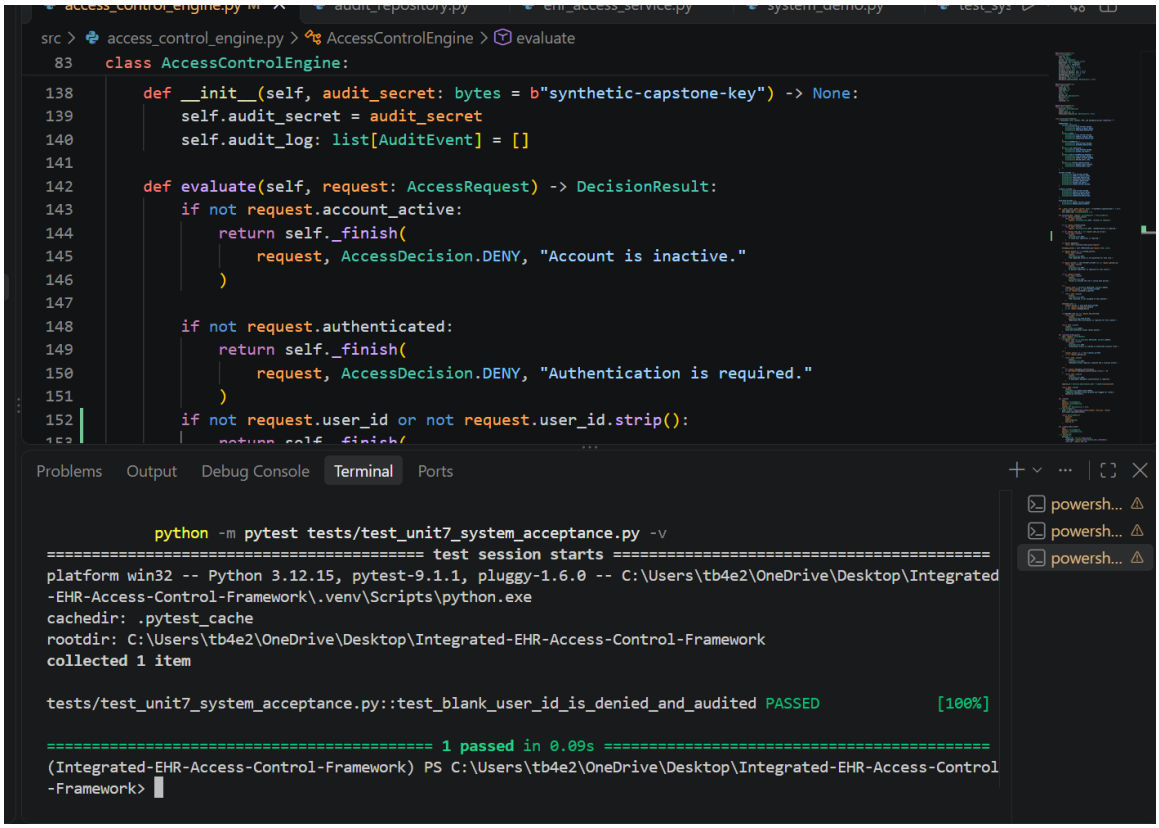
Note. The observed PERMIT contradicted the required DENY outcome and led directly to the validation correction.

The defect was resolved by checking both a missing value and user_id.strip() at the beginning of AccessControlEngine.evaluate, before role or emergency evaluation. The guard calls the existing _finish method, which records the denial and its reason, so the repair did not create an unlogged rejection path. The focused test then passed (Figure 6), and all 25 tests were executed to confirm that the stricter validation had not broken permitted, MFA, emergency, or persistence behavior (Figure 7). This fail–fix–retest sequence links a requirement, a discovered behavior, a code change, and a verified outcome, which is stronger evidence than a final green screen alone.

```
if not request.user_id or not request.user_id.strip():
    return self._finish(
        request,
        AccessDecision.DENY,
        "A valid user identifier is required.",
    )
```

Figure 6

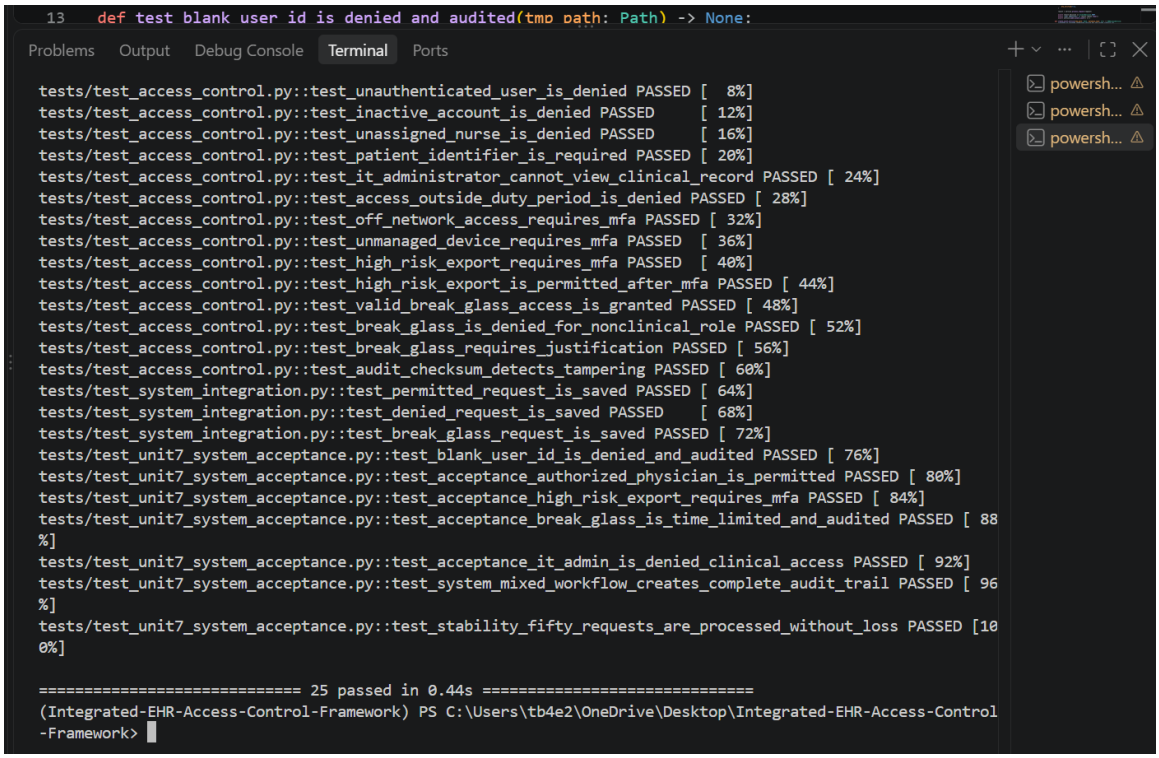
Corrected Validation and Focused Passing Test



Note. The engine denies a blank subject while preserving the standard audited return path; the targeted test reports one pass.

Figure 7

Complete 25-Test Regression Result



Note. Fifteen unit tests, three integration tests, and seven Unit 7 system, acceptance, and stability tests completed successfully in 0.44 seconds.

4.2.4 Scope and Limits of the Tests

The security-critical scope is deliberately limited. The suite verifies inactive, unauthenticated, and unidentified-subject denial, unauthorized roles, patient assignment, duty status, STEP_UP_MFA decisions, emergency justification, break-glass persistence, audit completeness, and detection of a modified signed event. It does not exercise a real identity provider, phishing or MFA bypass, stolen-key recovery, secret rotation, concurrent race conditions, network or TLS attacks, denial of service, dependency or container vulnerabilities, or penetration resistance; the fifty-request scenario is sequential, not a load test; and the acceptance cases are automated scenarios rather than sessions with clinicians. Accordingly, 25 passing tests establish conformance only for the stated synthetic cases, not defect-free code, production security, legal compliance, or clinical safety.

4.3 Evaluation Metrics

4.3.1 Benchmark Conditions

For reproducibility, the benchmark is a local sequential baseline obtained under documented conditions. Both runs used one Windows 11 workstation (build 10.0.26200) with a 12th-generation Intel Core processor (Intel64 Family 6 Model 154, 12 logical processors) and 7.7 GB of usable memory, running Python 3.12.15 in a single process; the final script records these values in results/evaluation_environment.json. Decision accuracy used the labeled cases described in Section 3.2.5: four in Unit 6 and 36 in the final run, six per role, each with an expected decision fixed before execution. The final script also counts false permits, meaning PERMIT or BREAK_GLASS_PERMIT returned where DENY or STEP_UP_MFA was expected. The performance block processed ten untimed warm-up requests and then 300 timed, sequential physician requests through the full service path, including the SQLite INSERT and commit for each request, against a temporary database file deleted afterward. In Unit 6 latency was the elapsed time divided by 300; in the final run each request was timed with time.perf_counter, so the mean, median, 95th percentile (p95), and 99th percentile (p99) are reported using the nearest-rank method. Throughput is 300 divided by the elapsed time of the timed block, and peak memory is the maximum incremental allocation reported by tracemalloc during that block. Operating-system activity, file caching, and laptop power state were not controlled, which is why repeated runs differ by a few percent and why none of the figures predicts a multiuser hospital deployment.

4.3.2 Quantitative and Qualitative Results

The evaluation script generated a CSV table, an environment record, and a PNG chart (Figure 8) on each run. In the Unit 6 run, decision accuracy was 100.00% (four of four cases), mean latency was 3.654–3.704 ms across two captures, throughput was 269.96–273.68 requests per second, peak incremental memory was 0.117 MB, and interface clarity was 4.60 out of 5. In the final run on October 24, 2026, all 36 labeled cases matched their expected decisions (100.00%) with zero false permits; mean latency was 3.196 ms, the median 3.131 ms, p95 3.976 ms, and p99 4.205 ms, within a range of 2.564–4.500 ms; throughput was 312.18 requests per second; and peak incremental memory was 0.127 MB. The decision path was not modified

between the runs, so the faster final figures reflect per-request timing and workstation variability rather than a code change. The distribution is the more informative result: the median and the p99 differ by about 1.1 ms and the slowest request took 4.5 ms, so no request approached the 250-ms design target. Table 9 reports both runs together with the expert validation summarized in Section 4.3.3.

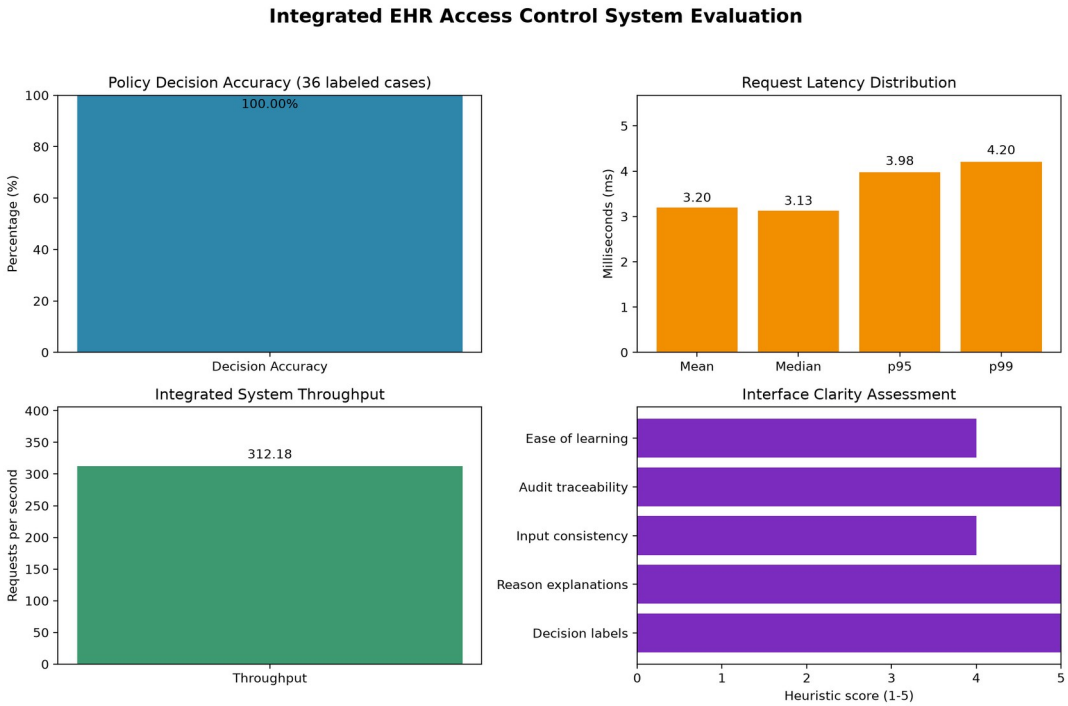
Table 9

Integrated Evaluation Results

Measure	Observed result	Interpretation	Status
Decision accuracy	100.00% (4/4) in Unit 6; 100.00% (36/36) with 0 false permits in the final run	Labeled decisions matched; no false permits	Target met (≥95%, zero false permits)
Mean latency	3.654–3.704 ms in Unit 6; 3.196 ms in the final run	Mean local integrated request time, including the SQLite write	Below the 250-ms design target
Median, p95, and p99 latency	Not recorded in Unit 6; 3.131, 3.976, and 4.205 ms in the final run	Distribution of the 300 timed requests; p99 about 60 times below target	NFR-01 met locally
Throughput	269.96–273.68 requests/s in Unit 6; 312.18 requests/s in the final run	Sequential local requests per second	Baseline established
Peak memory	0.117 MB in Unit 6; 0.127 MB in the final run	Peak incremental allocation during the timed block	Low local footprint
Interface clarity	4.60/5 (unchanged)	Developer heuristic across five stated criteria	Above the 4.0/5 target
Expert validation	4.85/5 (n = 2); external reviewer alone 4.80/5	Ten-statement review by two healthcare IT experts (Section 4.3.3)	Above the 4.0/5 target
Automated tests	25/25 passed on d2ac7cd and on 0bb3017	Unit, integration, system, acceptance, and stability	Complete pass

Figure 8

Integrated System Evaluation Chart



Note. The chart (results/system_evaluation_chart.png) reports decision accuracy over the 36 labeled cases, the latency distribution (mean, median, p95, p99), sequential throughput, and the five interface-clarity criteria from the final evaluation run on commit 0bb3017.

The clarity score was calculated from five stated criteria: decision labels, reason explanations, and audit traceability each scored 5, and input consistency and ease of learning each scored 4, giving a mean of 4.60. This makes the judgment inspectable and repeatable, but it remains a developer heuristic rather than a validated usability instrument; the expert validation reported next supplements it with independent practitioner judgement.

4.3.3 Expert Reviewer Validation

To validate the framework against practitioner judgement, two healthcare IT experts reviewed the prototype on October 24, 2026, using the structured instrument in Appendix D. Reviewer 1 is a senior healthcare information security consultant with 12 years of experience and has no connection to the author. Reviewer 2 is a senior health informatics and IT operations manager with 14 years of experience and is the author's brother; the relationship is disclosed here

in the interest of transparency. He had no role in the design, implementation, or testing of the framework, received only the public portfolio link and the same form as the external reviewer, and completed it without discussion with the author; his scores are reported separately in Table 10 so that the external reviewer's assessment can be read on its own. Each reviewer exercised the four demonstration scenarios and at least two variations of their own in the portfolio's in-browser decision console, which mirrors the released engine, then rated ten statements on a five-point Likert scale covering security behaviour (C1–C5), usability and clarity (C6–C9), and applicability (C10), gave open-ended feedback and a pilot recommendation, and signed a declaration that no real patient data had been shown.

Table 10

Expert Reviewer Ratings on the Ten Validation Statements

Statement	Reviewer 1	Reviewer 2	Mean
C1 Role permissions follow least privilege and match hospital work	5	5	5.00
C2 Contextual checks are appropriate for a hospital EHR	5	5	5.00
C3 Step-up MFA is required in the right situations	5	5	5.00
C4 Break-glass path is controlled appropriately	5	5	5.00
C5 Audit trail supports accountability; tamper detection adds value	5	5	5.00
C6 Decision labels are clear and unambiguous	4	5	4.50
C7 Decision reasons are understandable to non-technical staff	5	5	5.00
C8 Inputs are consistent and the system is easy to learn	5	4	4.50
C9 Audit traceability: who requested what, when, and the decision	5	5	5.00
C10 Applicable in a Saudi government hospital after development	4	5	4.50
Security behaviour (C1–C5)	5.00	5.00	5.00
Usability and clarity (C6–C9)	4.75	4.75	4.75
Applicability (C10)	4.00	5.00	4.50
Overall mean (C1–C10)	4.80	4.90	4.85

Note. Scale: 1 = strongly disagree, 5 = strongly agree. Success criterion: mean ≥ 4.0 with no unresolved critical concern (Table 2). Both reviewers answered the pilot question with "Yes, with the changes above."

Table 10 summarizes the ratings. Security behaviour received the maximum score from both reviewers (mean 5.00), usability and clarity averaged 4.75, and applicability averaged 4.50, giving an overall mean of 4.85 out of 5. This exceeds the success criterion of 4.0 set in the project proposal (Table 2), and the external reviewer's scores considered alone (mean 4.80) also exceed it, so Objective O5 is met. The two items that did not receive full marks were the decision labels (C6, one rating of 4) and ease of learning (C8, one rating of 4), which is consistent with the developer heuristic in Section 4.3.2, where ease of learning and input consistency were also the lowest-scoring dimensions. Both reviewers answered the pilot question with "Yes, with the changes above," and neither recorded a critical concern, that is, a rule that would block legitimate care or permit inappropriate access.

The open-ended feedback was consistent across the two reviewers. Both identified the same three strengths: least-privilege roles combined with clinical context, the controlled break-glass path with mandatory justification and a 15-minute expiry, and the tamper-evident audit trail, which Reviewer 2 related to regional accreditation and regulatory expectations. Their concerns were complementary. Reviewer 1 pointed to integration overhead with legacy EHR platforms and to possible latency from step-up MFA and context verification on a congested network, and recommended load and concurrency testing, alignment of authentication with national digital identity services such as Nafath, and an administrative dashboard for reviewing break-glass activations. Reviewer 2 pointed to the orientation clinicians would need to interpret step-up prompts during busy shifts and to the dependency on real-time directory services, and recommended caching for degraded-network continuity, role-specific micro-training within the interface, and export of audit events to standard SIEM connectors. These recommendations are carried into Sections 5.3 and 6.4, and the signed forms are reproduced in Appendix D.

4.3.4 Requirements Traceability and Objective Status

Table 11 links each requirement to the automated tests and other evidence that verify it, satisfying the 100% traceability criterion. Table 12 then reports the status of each objective

against the success criteria defined in Chapter 1; with the final evaluation and the expert validation complete, every objective is met.

Table 11

Requirements Traceability Matrix

ID	Verifying tests and evidence	Status
FR-01	test_assigned_physician_is_permitted; test_it_administrator_cannot_view_clinical_record; test_acceptance_it_admin_is_denied_clinical_access; role matrix (Table 4)	Met
FR-02	test_unauthenticated_user_is_denied; test_inactive_account_is_denied; test_blank_user_id_is_denied_and_audited; test_denied_request_is_saved	Met (after Unit 7 correction)
FR-03	test_off_network_access_requires_mfa; test_unmanaged_device_requires_mfa; test_high_risk_export_requires_mfa; test_high_risk_export_is_permitted_after_mfa; test_acceptance_high_risk_export_requires_mfa	Met
FR-04	test_unassigned_nurse_is_denied; test_patient_identifier_is_required; test_access_outside_duty_period_is_denied	Met
FR-05	test_valid_break_glass_access_is_granted; test_break_glass_is_denied_for_nonclinical_role; test_break_glass_requires_justification; test_break_glass_request_is_saved; test_acceptance_break_glass_is_time_limited_and_audited	Met
FR-06	test_audit_checksum_detects_tampering; test_permitted_request_is_saved; test_system_mixed_workflow_creates_complete_audit_trail; test_stability_fifty_requests_are_processed_without_loss	Met
NFR-01	Benchmark of 300 sequential requests with mean, median, p95, and p99 (Table 9); conditions and hardware in Section 4.3.1	Met locally (p99 4.205 ms against the 250-ms target)
NFR-02	25/25 regression on d2ac7cd and 0bb3017; maintenance schedule (Table 13)	Met for the prototype; restore drills planned
NFR-03	Dockerfile with non-root user, dependencies pinned to the verified versions, .env.example, branch and tag workflow	Met for the prototype

Table 12

Objective Status Against Success Criteria

Objective	Evidence	Status
O1: Least-privilege matrix	Six-role matrix implemented and tested; prohibited pairs denied	Met
O2: Testable requirements	Nine requirements with acceptance measures; 100% FR traceability	Met
O3: Executable prototype with signed evidence	Integrated service, SQLite persistence, HMAC verification, 25 tests	Met
O4: Multi-level testing and benchmark	25/25 tests; 100% accuracy on 36 labeled cases with zero false permits; documented benchmark with percentiles and hardware	Met
O5: Structured usability feedback	Heuristic 4.60/5; two-reviewer validation mean 4.85/5, external reviewer alone 4.80/5 (Table 10; Appendix D)	Met

4.4 Deployment and Version-Control Evidence

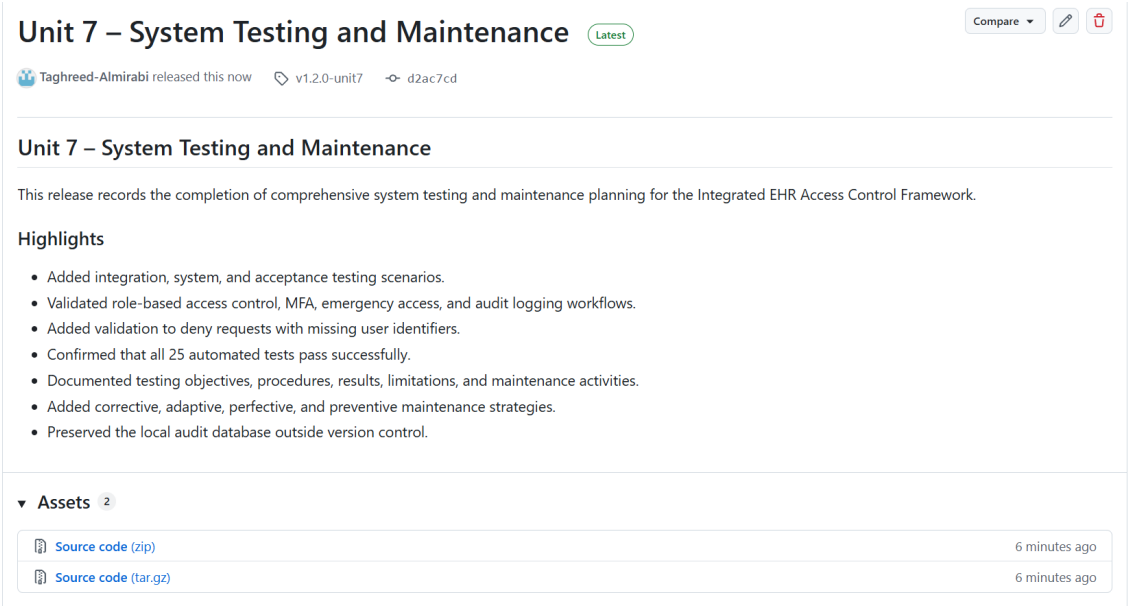
The deployment package includes requirements.txt, Dockerfile, .dockerignore, .env.example, and deployment/DEPLOYMENT.md. The Dockerfile installs pinned dependencies, copies the minimum application files, switches to the non-root ehruser account, declares /app/data as a volume, and runs system_demo.py. The sample environment file documents the intended configuration names—EHR_ENVIRONMENT, EHR_AUDIT_DB_PATH, EHR_LOG_LEVEL, EHR_AUDIT_SECRET, and EHR_USE_SYNTHETIC_DATA—and DEPLOYMENT.md explains their purpose. In the released prototype the database path and audit secret are supplied as constructor parameters with synthetic defaults; reading these values from the environment at the service entry point is recorded as an adaptive maintenance task so that the documented configuration boundary and the code converge.

Version control preserved traceability. The Unit 7 work was committed as ed9789c and f43e315 on the feature/unit7-system-testing-maintenance branch, merged into development (46e1e7d), and then into main at d2ac7cd; a clean checkout of d2ac7cd passed all 25 tests. The published release v1.2.0-unit7 targets d2ac7cd (Figure 9); the earlier v1.1.0-unit6 release targets

a4fbbfe, and v1.0.0 remains at cc2766a, the Unit 5 baseline. Release notes document the added tests, the validation correction, the synthetic-data scope, and the limitations. The tag-to-commit relationship closes the traceability gap identified in earlier feedback because the reported results correspond to an identifiable source state (GitHub, n.d.-b).

Figure 9

Verified Unit 7 GitHub Release

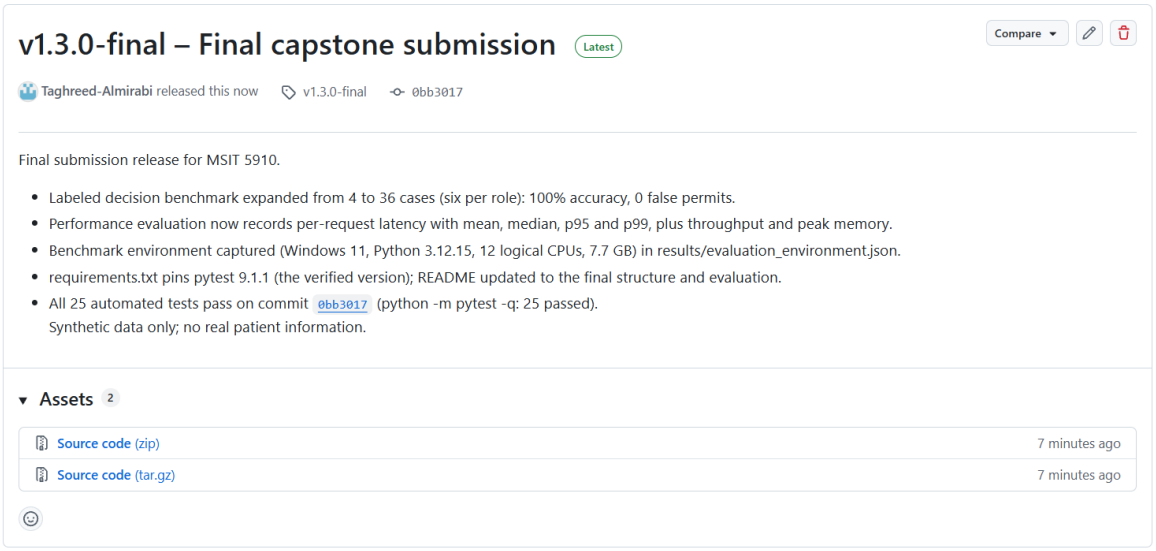


Note. The latest release links tag v1.2.0-unit7 to merge commit d2ac7cd and records the 25-test milestone.

The final submission added one more verified state. The evaluation-script, dependency-pin, README, and results changes were committed to main as 0bb3017 (0bb301769252c32b67086c8cf34520670baf0268) on October 24, 2026, after the complete suite had passed, and release v1.3.0-final was published from that commit with notes describing the 36-case benchmark, the latency percentiles, and the environment capture (Figure 10). Because the engine, service, and repository modules were not modified after d2ac7cd, the Unit 7 test evidence remains valid, the final results in Table 9 can be reproduced from the released source with the commands in Appendix A, and the repository offers two verified rollback points, each tied to a tag, a commit hash, and a recorded test result.

Figure 10

Verified Final Release v1.3.0-final



Note. The final release links tag v1.3.0-final to commit 0bb3017 and records the 36-case benchmark, the latency percentiles, the environment capture, the dependency alignment, and the 25-test result.

CHAPTER 5: DISCUSSION

5.1 Interpretation of Results

The results show an integrated, traceable request path: identity and context checks, emergency handling, audit integrity and persistence, tests, evaluation, and configuration work together. Each action yields an explained decision and a signed, saved event; the tests verify the path; and the deployment files document how to reproduce the environment. This answers RQ1 and RQ2 within the prototype boundary: stable roles and a small context set produced decisions that reviewers can read, and the prototype enforced normal, high-risk, and emergency decisions while producing complete, tamper-evident evidence.

All labeled evaluation scenarios matched the intended deterministic rules (100% policy accuracy), and the engine distinguished permits, denials, step-up MFA requirements, and emergency access. This supports internal validity because expected outputs were defined before comparison, but the labeled set (36 cases in the final run) measures rule conformance against a synthetic oracle rather than unknown hospital workflows, and the 25 automated tests provide complementary coverage of persistence and integrity. Local mean latency near 3 ms, a p99 of 4.2 ms, and sequential throughput above 300 requests per second indicate a lightweight authorization-and-persistence path well inside the 250-ms policy-decision design target. These are not hospital service-level results: the measurements exclude network, SSO, token validation, transit encryption, production database contention, log forwarding, backups, and concurrent clinical traffic, so the figures should be used only as a regression baseline, not as a capacity guarantee.

5.1.1 Quantitative Evaluation

Each metric addresses a different quality: accuracy tests labeled policy outcomes, latency measures one integrated request, throughput counts sequential requests per second, and peak memory measures additional allocation. Together they are more balanced than speed alone, consistent with the multiple quality characteristics of ISO/IEC 25010:2023 (ISO, 2023). Benchmark labels validate decisions, unit tests isolate branches, and integration tests verify database effects, so the layered evidence supports confidence in tested paths while leaving new

scenarios to reveal missing requirements. Reporting both runs avoids false precision: the difference between them (about 14% in mean latency) is larger than the 1–2% seen between the two Unit 6 captures, but both lie far below the target, and the final run adds the hardware record and the percentiles that the earlier report lacked. The percentiles are the more useful figures for a hospital reader, because a clinician experiences the slow requests rather than the average; with p99 at 4.2 ms, the slowest 1% of requests still completed about 60 times faster than the target.

5.1.2 Qualitative Evaluation and User Experience

Because a graphical clinical interface was out of scope, the heuristic rated command-line interaction. The decision labels, reasons, audit identifiers, and consistent request fields support clarity, which explains the 5/5 scores for labels, explanations, and traceability. Input consistency and ease of learning scored 4/5 because users must understand Python objects and flags; clinicians should not manually set assigned_to_patient or emergency, and a real EHR would source context from the session, the care assignment, the identity provider, and the emergency workflow. Security usability is a trade-off: excess warnings cause alert fatigue, while invisible controls invite workarounds. The expert validation (Section 4.3.3) partly addresses this gap from the practitioner side: both reviewers rated the reasons understandable to non-technical staff (C7, 5.00) and the labels clear (C6, 4.50), and both located the remaining friction where the heuristic did, in learning the inputs and interpreting step-up prompts during a busy shift. Later evaluation should include clinicians and HIM staff in approved workflow sessions, measuring task completion, errors, time, perceived effort, and interview feedback.

5.2 Comparison with Literature Findings

Cobrado et al.'s (2024) review found that EHR access-control research often lacks implementation detail, emergency access, MFA, consent, or accountability. This prototype does not address consent, but it combines the other gaps in one executable flow: RBAC provides baseline permissions, context limits role overreach, step-up MFA raises assurance, break-glass preserves availability, and signed audits support accountability. The approach is consistent with the NIST RBAC model's emphasis on administrable role structures (Ferraiolo et al., 2001) while avoiding the open-ended attribute policies that NIST's ABAC guidance warns are difficult to audit (Hu et al., 2014). Almaghrabi and Bugis (2022) identify privacy and confidentiality as

essential to reliable Saudi EHR use; role alone cannot stop legitimate staff from seeking data outside assigned duties, and contextual checks with complete logs narrow this gap, echoing the audit-trail evidence that clinicians' real access needs exceed static roles (Røstad & Edsberg, 2006).

Suleski and Ahmed (2023) support contextual healthcare authentication using user, environment, device, and use-case data, and NIST SP 800-63B-4 supports step-up authentication for higher-risk contexts (Temoshok et al., 2025); STEP_UP_MFA represents this principle without choosing a vendor, although production use still needs a real identity provider, phishing-resistant methods, and recovery. The break-glass design follows Brucker and Petritsch's (2009) principle that exceptional access should be an explicit, logged extension of the policy rather than a hidden override. NIST SP 800-207's zero-trust principle requires authentication and authorization before resource access rather than trust based on network location (Rose et al., 2020; Vukotich, 2023); the prototype evaluates each user's role, action, patient, and context even on a hypothetical hospital network, but it is not a complete zero-trust deployment because device posture, continuous risk signals, and enterprise policy infrastructure are absent.

Design choices map to NIST SP 800-53 families for access control, identification and authentication, audit and accountability, configuration management, and system integrity (Joint Task Force, 2020), to NIST log-management guidance (Kent & Souppaya, 2006), and to Saudi ECC-2:2024 (NCA, 2024); code alone does not establish compliance, which also requires policies, ownership, evidence, risk treatment, and monitoring. PDPL boundaries guided privacy design because storing personal data constitutes processing (SDAIA, 2023), so the prototype uses synthetic identifiers, and a pilot with realistic data would require authorization, documented purpose, access limits, retention rules, and privacy-impact review.

The reviewers' recommendations also echo the literature. Integration with national digital identity services is the practical form of the assurance-level guidance in NIST SP 800-63B-4 (Temoshok et al., 2025); the latency concern restates the availability tension that break-glass research addresses (Brucker & Petritsch, 2009); SIEM export and a break-glass review dashboard are the operational side of NIST SP 800-92 log-management guidance (Kent & Souppaya, 2006); and the training concern reflects the usability dimension that Cobrado et al. (2024) found under-reported in EHR access-control studies. That practitioners reached the same priorities from a demonstration is modest evidence that the design addresses the right problems.

5.3 Implications and Limitations

Roles and scenarios came from the literature and project assumptions, not from an approved hospital policy catalog. Departments, specialties, delegation, trainees, researchers, external providers, consent, and temporary assignments complicate real authorization, so the six-role model is incomplete and stakeholders must validate rules against actual workflows before results can generalize. Two variances from the Unit 3 plan were closed for the final submission and one remains. The labeled scenario set was expanded from four to the planned 36 cases and the two-reviewer validation was conducted with the instrument in Appendix D, so O4 and O5 are reported as met; automated joiner–mover–leaver processing is still designed but represented only by account-state and duty flags. The validation itself has limits. Two reviewers confirm face validity but are too few for statistical generalisation, and a larger panel including practising clinicians and a hospital compliance officer would strengthen the usability findings. One reviewer is a family member of the author; although he worked only from the public portfolio and the standard form and had no part in the project, this is a potential source of bias, so the relationship is disclosed in Section 4.3.3 and the external reviewer's scores (mean 4.80, also above the target) are reported separately. Both reviewers judged the prototype from its demonstration scenarios rather than a live workflow, so their applicability ratings (mean 4.50) are an informed projection, and they identified dependencies the prototype does not yet address: legacy EHR and directory integration, behaviour under network degradation, and clinician training. Evaluation used one workstation, one Python process, and SQLite; tests did not cover identity integration, credential attacks, stolen keys, concurrency, network controls, vulnerability scans, or penetration tests; the benchmark reports percentiles but not confidence intervals from repeated trials; and the 4.60/5 clarity score remains a developer heuristic. Findings are reproducible at this stage but cannot establish hospital capacity, security authorization, clinical safety, or user acceptance.

5.3.1 Deployment Strategy

Options were direct on-premise Python, managed cloud, or containers. Direct installation risks environment drift; cloud requires approved data residency, contractual control, identity integration, and regulatory oversight; Docker packages the application for approved infrastructure without dictating the hosting location. Docker on a hospital-controlled Linux

server or an approved Saudi private cloud therefore offers reproducibility and control. The image uses a slim Python base, pinned dependencies, a non-root user, and a persistent volume, following Docker's recommendations for trusted small base images, limited build context, and image testing in continuous integration (Docker, n.d.). Before production, the image should be pinned by digest, scanned, signed, stored in an approved registry, and promoted through development, test, staging, and production, and rollout should be incremental: synthetic data first, then a nonproduction identity provider and a masked EHR, then production-like staging with a managed database, centralized logs, backups, and performance tests, and finally a read-only pilot in selected departments with monitoring and rollback.

5.3.2 Configuration Management and Reliability

Explicit configuration improves reliability. The documented variables identify the deployment stage, the audit database path, the logging level, the signing key, and the synthetic-data mode; .env.example lists the names without production secrets, requirements.txt pins packages, the Dockerfile defines the runtime, and DEPLOYMENT.md records commands and cautions. Wiring the variables into the service entry point will make that baseline authoritative for the running code. The placeholder audit key must never be used in production: secrets should be injected through a secrets manager, restricted by service identity, rotated under procedure, and never logged, and the service should fail fast if required production settings are missing. SQLite suits a single-user demonstration, not production auditing; a managed relational database should provide encryption at rest, high availability, access controls, monitored backups, point-in-time recovery, and retention, with transactional audit writes and defined behavior on failure. GitHub Actions could run pytest, scan dependencies, and build containers on every change (GitHub, n.d.-a), protected branches can block untested changes to main, and the published release links the tag and release notes to a reproducible rollback point (GitHub, n.d.-b).

5.3.3 Security, Privacy, and Ethical Considerations

Least-privilege roles reduce unnecessary actions; context prevents universal access; step-up MFA strengthens sensitive activity; break-glass preserves emergency access with a visible exception; and HMAC supports integrity checks, while logging denials as well as permits can reveal misconfiguration, training needs, or probing. Break-glass balances patient safety against privacy and insider abuse: justification, expiration, identification, and persistent audit mitigate

the risk, and production should notify a supervisor or privacy officer, require retrospective review within a defined period, detect repeated use, and sanction misuse. No real patient data were used, which minimized privacy risk and made consent unnecessary for this technical demonstration, but duties remain: synthetic identifiers are labeled as such, screenshots contain no credentials, secrets, or patient data, repository visibility is reviewed, and the reviewers gave informed consent by signing the declaration on the evaluation form. Any realistic data must be lawfully sourced, de-identified, minimized, access-controlled, and retained only as needed.

5.4 Maintenance and Operational Sustainability

The Unit 7 results make maintenance part of the security model. Corrective maintenance addresses confirmed defects, as demonstrated by the blank-identifier fix; adaptive maintenance responds to dependency changes, identity-provider interfaces, EHR APIs, deployment platforms, revised Saudi requirements, and the environment-variable wiring described above, and the alignment of the pytest pin to the verified 9.1.1 in release v1.3.0-final is a small completed example; perfective maintenance improves explanations, accessibility, policy administration, performance, and clinician workflows; and preventive maintenance includes refactoring, dependency and container scanning, key-rotation exercises, access recertification, and restore testing (ISO, 2022a; Thales, n.d.). Change management begins with an issue that records severity, affected requirement, reproduction evidence, acceptance criteria, and owner; a feature or hotfix branch carries the code, tests, configuration, and documentation changes for one reviewable unit; and automated tests must pass before merge. A failed security or access-control check blocks the merge, is reported in the pull request with the failing requirement and scenario, and is remediated on the same branch before retest. Protected tags identify immutable milestones, with the verified Unit 7 tag as a rollback point, supporting the NIST SSDF emphasis on repeatable secure-development evidence and NIST SP 800-53 configuration-management, audit, access-control, and system-integrity controls (Joint Task Force, 2020; Souppaya et al., 2022). Table 13 presents the proposed schedule.

Table 13

Proposed Maintenance and Monitoring Schedule

Frequency	Activity	Primary evidence
Every commit or pull request	Run the full test suite; review policy, schema, dependency, and configuration changes.	CI log, review record, linked issue
Weekly	Review authentication failures, denials, emergency events, checksum alerts, and open defects.	Operations and security review
Monthly	Patch dependencies; scan source and image; compare errors, latency, memory, and storage growth.	Patch and metric report
Quarterly	Recertify roles; test backup restore, secret rotation, and rollback; review break-glass use.	Signed governance record
Annually or after major change	Refresh threat model, privacy impact assessment, architecture, recovery objectives, and user training.	Multidisciplinary approval

5.5 Post-Deployment Risks and Mitigation

Table 14 is the project's risk register for the post-deployment stage. Each risk is paired with its likely effect and with mitigation or contingency measures that map to the maintenance schedule and the configuration-management rules above.

Table 14

Post-Deployment Risks and Controls

Risk	Likely effect	Mitigation and contingency
Vulnerable dependency or image	Unauthorized access or service compromise	Scanning, risk-based patch timelines, signed image, incident response, rollback to the last verified release
Policy or privilege drift	Excessive access or false denial	Versioned policy, separation of duties, quarterly recertification, regression cases
Audit inconsistency, tampering, or growth	Missing evidence, failed writes, exhausted storage	Transactional store, checksum verification, centralized logging, retention, alerts, backup and restore drills
Emergency-access misuse	Privacy breach or insider abuse	Justification, expiry, alert, retrospective review, trend monitoring, sanctions
Configuration drift	Different behavior across environments	Approved baseline, secrets manager, staged promotion, health checks, tested rollback
Usability-driven workarounds	Shared credentials, bypass attempts, care delay	Clinician testing, clear reasons, training, support metrics, controlled rule refinement
Directory or network dependency (reviewer finding)	Context checks unavailable or slow; decisions delayed during network degradation	Cached role and assignment context with an audited cache window; fail-closed default with the approved downtime and emergency procedure; latency percentiles tracked per release

5.6 Overall Interpretation

Taken together, the evidence supports a modest but useful conclusion. The prototype is internally coherent: modules exchange the expected objects, decisions are persisted, the stated test cases pass, the discovered defect was corrected without regression, and local resource use is small. The result is stronger than a diagram-only proposal because the architecture has executable behavior, release traceability, and maintenance evidence; it remains weaker than a hospital pilot because identity, clinical workflow, infrastructure, and human factors are simulated. In answer to RQ3, the appropriate next decision is controlled advancement to a nonproduction integration environment, now supported by the reviewers' pilot recommendation and shaped by the conditions they attached to it, not unrestricted deployment.

CHAPTER 6: CONCLUSION AND FUTURE WORK

6.1 Summary of Main Findings

The capstone produced an EHR access-control prototype for a modeled Saudi government hospital, joining six-role RBAC, patient assignment and duty context, step-up MFA, break-glass access, HMAC-signed audits, SQLite persistence, an application service, automated tests at four levels, a performance evaluation, deployment documentation, and a maintenance plan. All 25 tests passed on both released commits; 36 labeled scenarios reached 100% policy accuracy with no false permits; the final benchmark measured 3.196 ms mean and 4.205 ms p99 latency and 312.18 requests per second with 0.127 MB peak memory; interface clarity scored 4.60/5; and two healthcare IT experts rated the framework 4.85/5 and recommended a controlled pilot. One system test exposed a blank-identifier defect that was corrected and verified without regression. The integrated design produces reviewable outcomes: patient context narrows role access, step-up MFA strengthens assurance for risky activity, emergency access stays attributable, and each request creates a retrievable audit record, while Git branches, merges, tags, and releases preserved development traceability.

6.2 Implications and Project Contribution

The project's contribution is a practical bridge between security recommendations and an executable healthcare example. The literature calls for access control that accounts for authentication, emergency care, and accountability, but these elements are often described separately; this prototype demonstrates how they can cooperate without a complete EHR implementation, and it offers a reusable evaluation pattern: define labeled policy scenarios, verify persistence with integration tests, measure local behavior under documented conditions, trace every requirement to a test, and document a realistic deployment and maintenance path. For the Saudi setting, the project demonstrates privacy by design through synthetic data, least privilege, purpose-related context, and auditable exceptions, consistent with the direction of the

PDPL and the national cybersecurity controls, although the prototype is not presented as certified or compliant. For practitioners, a governed break-glass path shows that security and clinical availability need not be opposites; for researchers, the prototype provides a baseline that can be expanded with consent handling, adaptive authentication, policy administration, and stakeholder evaluation.

6.3 Lessons Learned and Challenges

The most important technical lesson was that integration exposes assumptions that unit code can hide: the original engine could decide and log in memory, but the final system needed a clear transaction boundary, durable storage, retrieval methods, repeatable configuration, and proof that an emergency decision reached the database, and creating EHRAccessService solved this without duplicating policy logic. A second lesson was that measurement must be explained; the slight difference between two benchmark runs showed why ranges, repeated trials, and environmental details matter. A third lesson came from Unit 7: a final system test must challenge assumptions that unit tests may not express, as the whitespace-only identifier passed authentication flags and role checks until a complete scenario asked whether a decision could be attributed to a valid subject. A green test count is meaningful only when its scope, environment, failure history, and source version are documented. Practical challenges included keeping repository structure and imports consistent across source, tests, and scripts; isolating database tests with temporary paths; handling locked performance output files; separating deployment settings from code while recording the remaining gap between documented and consumed configuration; and isolating each unit's work on a feature branch that was reviewed, merged, and tagged. A fourth lesson came from the expert validation: the practitioners did not question the rules, which both scored at the maximum, but the operating conditions around them, so a security design is judged in practice by how it behaves when its inputs are late, missing, or misunderstood; the final risk register and future work reflect that.

6.4 Future Work

The three improvements that the Unit 7 report promised for the final submission, the reviewer validation, the 36-case benchmark, and percentile latency with hardware details, have

been delivered, so the future work concerns the next engineering stage, and the reviewers' recommendations set its first priorities: load and concurrency testing of the decision path under emergency-department volumes; integration of authentication with national digital identity services such as Nafath at the assurance level defined in SP 800-63B-4; an administrative review dashboard for break-glass activations with a defined triage time; local caching of role and assignment context, with the cache window itself audited, so that decisions continue during directory or network degradation; role-specific in-interface guidance that explains step-up prompts and contextual denials; and export of audit events to standard SIEM connectors for hospital security operations. Beyond those priorities, the next stage should expose the service behind an authenticated API in an isolated environment, integrate a nonproduction standards-based identity provider, read configuration from the documented environment variables, replace hard-coded policy data with an approved versioned policy store, model consent where legally and clinically relevant, and move audit records to an encrypted managed database with centralized monitoring that alerts on emergency access, repeated denials, and checksum failures. Testing should add concurrent clients, latency percentiles, failure injection, malformed requests, key rotation, property-based role–action–context combinations with invariants such as "unauthenticated requests are never permitted," dependency and container scans, static analysis, and independent penetration testing. Clinicians, HIM staff, privacy professionals, and security reviewers should complete approved workflow sessions using a validated usability instrument, and deployment should use signed, scanned images and a controlled CI/CD promotion workflow, with monitoring, recovery, incident response, key rotation, rollback, and break-glass review documented before any contact with real patient data.

6.5 Final Conclusion

The project met its academic objective: it transformed an access-control proposal into a working, testable, measurable, and maintainable prototype packaged for controlled deployment while preserving a responsible scope. Its strongest result is the traceable chain from request, through an explainable decision, to persisted audit evidence and a verified release; its strongest limitation is that the evidence comes from a synthetic, single-machine environment and from a two-person expert review rather than from clinicians in a live workflow. By reporting both

achievements and boundaries, the capstone provides a credible foundation for future development: the prototype should be treated as an evaluated design artifact, useful for stakeholder review and controlled experimentation, but requiring substantial security, privacy, usability, and operational validation before any contact with real patient data.

REFERENCES

Almaghrabi, N. S., & Bugis, B. A. (2022). Patient confidentiality of electronic health records: A recent review of the Saudi literature. *Dr. Sulaiman Al Habib Medical Journal, 4*(3), 126–135. https://doi.org/10.1007/s44229-022-00016-9

Brucker, A. D., & Petritsch, H. (2009). Extending access control models with break-glass. In *Proceedings of the 14th ACM Symposium on Access Control Models and Technologies* (pp. 197–206). ACM. https://doi.org/10.1145/1542207.1542239

Cobrado, U. N., Sharief, S., Regahal, N. G., Zepka, E., Mamauag, M., & Velasco, L. C. (2024). Access control solutions in electronic health record systems: A systematic review. *Informatics in Medicine Unlocked, 49*, Article 101552. https://doi.org/10.1016/j.imu.2024.101552

Docker. (n.d.). *Building best practices*. Docker Docs. Retrieved October 20, 2026, from https://docs.docker.com/build/building/best-practices/

Fernández-Alemán, J. L., Señor, I. C., Lozoya, P. Á. O., & Toval, A. (2013). Security and privacy in electronic health records: A systematic literature review. *Journal of Biomedical Informatics, 46*(3), 541–562. https://doi.org/10.1016/j.jbi.2012.12.003

Ferraiolo, D. F., Sandhu, R., Gavrila, S., Kuhn, D. R., & Chandramouli, R. (2001). Proposed NIST standard for role-based access control. *ACM Transactions on Information and System Security, 4*(3), 224–274. https://doi.org/10.1145/501978.501980

GitHub. (n.d.-a). *GitHub Actions documentation*. GitHub Docs. Retrieved October 20, 2026, from https://docs.github.com/en/actions

GitHub. (n.d.-b). *Managing releases in a repository*. GitHub Docs. Retrieved October 20, 2026, from https://docs.github.com/en/repositories/releasing-projects-on-github/managing-releases-in-a-repository

Hu, V. C., Ferraiolo, D., Kuhn, R., Schnitzer, A., Sandlin, K., Miller, R., & Scarfone, K. (2014). *Guide to attribute based access control (ABAC) definition and considerations* (NIST

Special Publication 800-162). National Institute of Standards and Technology. https://doi.org/10.6028/NIST.SP.800-162

International Organization for Standardization. (2022a). *Software engineering—Software life cycle processes—Maintenance* (ISO/IEC/IEEE Standard No. 14764:2022). https://www.iso.org/standard/80710.html

International Organization for Standardization. (2022b). *Software and systems engineering—Software testing—Part 1: General concepts* (ISO/IEC/IEEE Standard No. 29119-1:2022). https://www.iso.org/standard/81291.html

International Organization for Standardization. (2023). *Systems and software engineering—Systems and software quality requirements and evaluation (SQuaRE)—Product quality model* (ISO/IEC Standard No. 25010:2023). https://www.iso.org/standard/78176.html

Joint Task Force. (2020). *Security and privacy controls for information systems and organizations* (NIST Special Publication 800-53, Revision 5). National Institute of Standards and Technology. https://doi.org/10.6028/NIST.SP.800-53r5

Kent, K., & Souppaya, M. (2006). *Guide to computer security log management* (NIST Special Publication 800-92). National Institute of Standards and Technology. https://doi.org/10.6028/NIST.SP.800-92

Letaw, L. (2024). Requirements. In *Handbook of software engineering methods*. Oregon State University. https://open.oregonstate.education/setextbook/chapter/requirements/

Martyniuk, I. (2026, August 31). *What is system testing? Process, best practices and common misconceptions*. TestFort. https://testfort.com/blog/what-is-system-testing-in-software-testing

National Cybersecurity Authority. (2024). *Essential cybersecurity controls* (ECC-2:2024). https://nca.gov.sa/en/regulatory-documents/controls-list/ecc/

Oguz, A. (2025). *Project management* (2nd ed.). MSL Academic Endeavors. https://pressbooks.ulib.csuohio.edu/projectmanagement2ndedition/

OWASP Foundation. (2025). *OWASP Top 10:2025*. https://owasp.org/www-project-top-ten/

pytest-dev team. (n.d.). *pytest documentation*. Retrieved October 20, 2026, from https://docs.pytest.org/en/stable/

Python Software Foundation. (n.d.-a). *hmac—Keyed-hashing for message authentication* (Python 3.12 documentation). Retrieved October 20, 2026, from https://docs.python.org/3.12/library/hmac.html

Python Software Foundation. (n.d.-b). *sqlite3—DB-API 2.0 interface for SQLite databases* (Python 3.12 documentation). Retrieved October 20, 2026, from https://docs.python.org/3.12/library/sqlite3.html

Python Software Foundation. (n.d.-c). *tracemalloc—Trace memory allocations* (Python 3.12 documentation). Retrieved October 20, 2026, from https://docs.python.org/3.12/library/tracemalloc.html

Rose, S., Borchert, O., Mitchell, S., & Connelly, S. (2020). *Zero trust architecture* (NIST Special Publication 800-207). National Institute of Standards and Technology. https://doi.org/10.6028/NIST.SP.800-207

Røstad, L., & Edsberg, O. (2006). A study of access control requirements for healthcare systems based on audit trails from access logs. In *Proceedings of the 22nd Annual Computer Security Applications Conference* (pp. 175–186). IEEE. https://doi.org/10.1109/ACSAC.2006.8

Saudi Data and Artificial Intelligence Authority. (2023). *Guide to the Saudi Personal Data Protection Law for controllers and processors*. https://dgp.sdaia.gov.sa/wps/portal/pdp/knowledgecenter/details/PDPLCP

Souppaya, M., Scarfone, K., & Dodson, D. (2022). *Secure Software Development Framework (SSDF) version 1.1: Recommendations for mitigating the risk of software vulnerabilities* (NIST Special Publication 800-218). National Institute of Standards and Technology. https://doi.org/10.6028/NIST.SP.800-218

Suleski, T., & Ahmed, M. (2023). A data taxonomy for adaptive multifactor authentication in the Internet of Health Care Things. *Journal of Medical Internet Research, 25*, Article e44114. https://doi.org/10.2196/44114

Suleski, T., Ahmed, M., Yang, W., & Wang, E. (2023). A review of multi-factor authentication in the Internet of Healthcare Things. *Digital Health*, *9*, 1–20. https://doi.org/10.1177/20552076231177144

Summers, B. L. (2020). *Effective methods for software engineering*. CRC Press. https://doi.org/10.1201/9781003025665

Temoshok, D., Fenton, J. L., Choong, Y.-Y., Lefkovitz, N., Regenscheid, A., Galluzzo, R., & Richer, J. P. (2025). *Digital identity guidelines: Authentication and authenticator management* (NIST Special Publication 800-63B-4). National Institute of Standards and Technology. https://doi.org/10.6028/NIST.SP.800-63B-4

Thales. (n.d.). *What is a software maintenance process? 4 types of software maintenance*. Retrieved October 20, 2026, from https://cpl.thalesgroup.com/software-monetization/four-types-of-software-maintenance

Vukotich, G. (2023). Healthcare and cybersecurity: Taking a zero trust approach. *Health Services Insights*, *16*. https://doi.org/10.1177/11786329231187826

APPENDICES

Appendix A: Test Commands and Evidence Map

The following commands reproduce the Unit 7 and final-submission evidence from the repository root after installing the pinned dependencies in a virtual environment.

```
python -m pytest tests/test_unit7_system_acceptance.py -v
python -m pytest -v > results/unit7_full_test_log.txt
python -m pytest -q
python system_demo.py
python performance_evaluation.py
```

- tests/test_access_control.py: 15 policy-focused unit tests.
- tests/test_system_integration.py: three service–repository integration tests.
- tests/test_unit7_system_acceptance.py: seven system, acceptance, and stability tests (Table 8).
- results/unit7_full_test_log.txt: verbose 25-test execution record (Windows, Python 3.12.15, pytest 9.1.1).
- results/evaluation_results.csv, results/evaluation_environment.json, and results/system_evaluation_chart.png: saved outputs of the final evaluation run (36 labeled cases, latency percentiles, hardware record).
- Figures 5–7: initial failure, corrected focused run, and final regression result.

Appendix B: Configuration and Deployment Summary

The documented configuration names and local commands are reproduced below; the local SQLite path and synthetic-data setting are demonstration defaults only.

```
EHR_ENVIRONMENT=development
EHR_AUDIT_DB_PATH=data/ehr_audit.db
EHR_LOG_LEVEL=INFO
EHR_AUDIT_SECRET=<inject from an approved secrets manager>
EHR_USE_SYNTHETIC_DATA=true

python -m venv .venv
python -m pip install -r requirements.txt
python -m pytest -q
docker build -t integrated-ehr-access-control:1.2.0 .
docker run --rm --env-file .env -v ehr-audit-data:/app/data \
    integrated-ehr-access-control:1.2.0
```

Production deployment must replace the example secret, use an approved identity provider and managed database, enforce TLS, centralize monitoring, scan and sign the image, and complete governance approval.

Appendix C: Repository and Release

Repository: https://github.com/Taghreed-Almirabi/Integrated-EHR-Access-Control-Framework

Verified Unit 7 release: https://github.com/Taghreed-Almirabi/Integrated-EHR-Access-Control-Framework/releases/tag/v1.2.0-unit7

Final release: https://github.com/Taghreed-Almirabi/Integrated-EHR-Access-Control-Framework/releases/tag/v1.3.0-final — commit 0bb3017 (0bb301769252c32b67086c8cf34520670baf0268), committed to main on October 24, 2026, after the complete suite passed (25/25).

Project portfolio (GitHub Pages): https://taghreed-almirabi.github.io/capstone-portfolio/

Verified Unit 7 merge commit: d2ac7cd (d2ac7cdd4ae9c99217a074affd515bb8a63298a9), merged into main on October 17, 2026. Earlier milestones: v1.1.0-unit6 (a4fbbfe), v1.0.0 (cc2766a), v0.2.0-tested (5a7e9ab), and v0.1.0-core (76910c7).

Appendix D: Expert Reviewer Validation Instrument and Completed Forms

Two healthcare IT experts completed the instrument below on October 24, 2026 (Section 4.3.3). Each reviewer received the public portfolio link, exercised the demonstration scenarios in the browser, rated the ten statements C1–C10 shown on page 2 of the form from 1 (strongly

disagree) to 5 (strongly agree), answered open questions on strengths, weaknesses, and improvements, stated whether a hospital pilot is recommended, and signed a declaration that the evaluation reflected independent judgement and that no real patient data had been shown. The success threshold was a mean of at least 4.0 with no unresolved critical concern.

The completed and signed forms are reproduced below with the reviewers' permission (Figures D1–D6). Reviewer 1 is the external reviewer; Reviewer 2 is the author's brother, as disclosed in Section 4.3.3.

Figure D1

Reviewer 1 Evaluation Form, Page 1 (Information and Review Steps)

Appendix D

Expert Reviewer Evaluation Form

Designing and Evaluating an Integrated Access Control Framework for Electronic Health Records in a Saudi Government Hospital

Taghreed Y. Almirabi | MSIT 5910-01 Capstone Project | University of the People | Supervisor: Dr. Stella T. Whyte

Thank you for reviewing this academic prototype. The review takes about 20 minutes. The system uses synthetic users and patients only; no real patient information is involved. Your name and role will appear in the capstone report only with your permission, and your ratings will be reported as aggregated scores.

Section A. Reviewer information

Full name	Basem Siddiq Ali
Job title / role	Senior Healthcare Information Security Consultant
Organization (optional)	Healthcare IT Security Advisory
Years of experience in healthcare IT or information security	12 years
Date of review	October 24, 2026

Section B. What to review

Please complete the following steps before rating. No installation is needed; everything runs in your web browser.

1. Open the project portfolio: https://taghreed-almirabi.github.io/capstone-portfolio/
2. In the panel “Try an access request”, click each of the four scenario buttons and read the decision, the reason and the audit line that appear below the form.
3. Change at least two settings yourself. For example, untick “On duty” and watch the decision change; tick “Emergency (break-glass)” and type a justification; select “IT systems administrator” and request “View patient record”.
4. Read the section “How a request is decided” and the role matrix.
5. Optional: browse the “Evidence” section (test suite, measured results) and the “Deliverables” section (source code and documentation).

Note. Signed evaluation form reproduced with the reviewer's permission; synthetic data only.

Figure D2

Reviewer 1 Evaluation Form, Page 2 (Ratings)

Section C. Ratings

Mark one box per statement with an X. Scale: 1 = Strongly disagree, 2 = Disagree, 3 = Neutral, 4 = Agree, 5 = Strongly agree.

No.	Statement	1	2	3	4	5
Security behaviour						
C1	The role permissions follow least privilege and match how a hospital actually works.					X
C2	The contextual checks (duty period, patient assignment, network, device) are appropriate for a hospital EHR.					X
C3	Step-up MFA is required in the right situations (record export, account management, off-network access, unmanaged device).					X
C4	The break-glass emergency path is controlled appropriately (clinical roles only, written justification, 15-minute expiry, flagged for review).					X
C5	The audit trail gives enough information for accountability and investigation, and tamper detection adds real value.					X
Usability and clarity						
C6	The decision labels (PERMIT, DENY, STEP_UP_MFA, BREAK_GLASS_PERMIT) are clear and unambiguous.				X	
C7	The reason given for each decision would be understandable to a non-technical clinician or administrator.					X
C8	The inputs are consistent and the system is easy to learn.					X
C9	Audit traceability: I can tell who requested what, when, and what was decided.					X
Overall						
C10	With further development, this framework is applicable in a real Saudi government hospital environment.				X	

Note. Signed evaluation form reproduced with the reviewer's permission; synthetic data only.

Figure D3

Reviewer 1 Evaluation Form, Page 3 (Open Feedback and Declaration)

Section D. Open feedback

D1. Main strengths of the framework (two or three points)
1. Comprehensive RBAC & Contextual Integration: Excellent implementation of the principle of least privilege combined with dynamic contextual parameters (duty status, patient assignment, device trust, secure network location) tailored specifically to clinical workflows. 2. Secure Break-Glass Emergency Mechanism: Well-structured emergency override path balancing urgent patient care necessities with strict accountability through mandatory justification, time-bound access (15-minute expiry), and instant audit flagging. 3. Accountability & Integrity: The inclusion of tamper-evident audit logging provides high traceability and satisfies essential healthcare compliance requirements.

D2. Weaknesses, risks or gaps you noticed
1. Integration Complexity with Legacy Systems: Integrating this framework with legacy Electronic Health Record (EHR) platforms and proprietary hospital systems deployed across Saudi hospitals may present technical overhead. 2. Latency during Peak Hours: Real-time multi-factor authentication (MFA) step-up triggers and context verification could introduce latency during emergency situations if network infrastructure experiences bottlenecks.

D3. Suggested improvements before a hospital pilot
1. Conduct rigorous concurrency and latency load testing to ensure real-time responsiveness during high-pressure emergency department shifts. 2. Align the authentication module with national Saudi digital identity standards (e.g., integration with NAFATH / unified governmental single sign-on frameworks). 3. Build an advanced administrative dashboard for security officers to streamline post-incident reviews of break-glass activations.

D4. Would you recommend piloting this framework in a hospital setting after further development? Yes / [X] Yes, with the changes above / No

Section E. Declaration

I confirm that this evaluation reflects my independent professional judgement, that I reviewed the prototype described in Section B, and that no real patient data was shown to me.

Reviewer signature	Date
Basem Siddiq Ali	October 24, 2026

Please return the completed form to the student by email. Thank you for your time and expertise.

Note. Signed evaluation form reproduced with the reviewer's permission; synthetic data only.

Figure D4

Reviewer 2 Evaluation Form, Page 1 (Information and Review Steps)

Appendix D

Expert Reviewer Evaluation Form

Designing and Evaluating an Integrated Access Control Framework for Electronic Health Records in a Saudi Government Hospital

Taghreed Y. Almirabi | MSIT 5910-01 Capstone Project | University of the People | Supervisor: Dr. Stella T. Whyte

Thank you for reviewing this academic prototype. The review takes about 20 minutes. The system uses synthetic users and patients only; no real patient information is involved. Your name and role will appear in the capstone report only with your permission, and your ratings will be reported as aggregated scores.

Section A. Reviewer information

Full name	Abdulrahim Yousif Almirabi
Job title / role	Senior Health Informatics & IT Operations Manager
Organization (optional)	Saudi Health Digital Transformation Directorate
Years of experience in healthcare IT or information security	14 years
Date of review	October 24, 2026

Section B. What to review

Please complete the following steps before rating. No installation is needed; everything runs in your web browser.

1. Open the project portfolio: https://taghreed-almirabi.github.io/capstone-portfolio/
2. In the panel “Try an access request”, click each of the four scenario buttons and read the decision, the reason and the audit line that appear below the form.
3. Change at least two settings yourself. For example, untick “On duty” and watch the decision change; tick “Emergency (break-glass)” and type a justification; select “IT systems administrator” and request “View patient record”.
4. Read the section “How a request is decided” and the role matrix.
5. Optional: browse the “Evidence” section (test suite, measured results) and the “Deliverables” section (source code and documentation).

Note. Signed evaluation form reproduced with the reviewer's permission; synthetic data only.

Figure D5

Reviewer 2 Evaluation Form, Page 2 (Ratings)

Section C. Ratings

Mark one box per statement with an X. Scale: 1 = Strongly disagree, 2 = Disagree, 3 = Neutral, 4 = Agree, 5 = Strongly agree.

No.	Statement	1	2	3	4	5
Security behaviour						
C1	The role permissions follow least privilege and match how a hospital actually works.					X
C2	The contextual checks (duty period, patient assignment, network, device) are appropriate for a hospital EHR.					X
C3	Step-up MFA is required in the right situations (record export, account management, off-network access, unmanaged device).					X
C4	The break-glass emergency path is controlled appropriately (clinical roles only, written justification, 15-minute expiry, flagged for review).					X
C5	The audit trail gives enough information for accountability and investigation, and tamper detection adds real value.					X
Usability and clarity						
C6	The decision labels (PERMIT, DENY, STEP_UP_MFA, BREAK_GLASS_PERMIT) are clear and unambiguous.					X
C7	The reason given for each decision would be understandable to a non-technical clinician or administrator.					X
C8	The inputs are consistent and the system is easy to learn.				X	
C9	Audit traceability: I can tell who requested what, when, and what was decided.					X
Overall						
C10	With further development, this framework is applicable in a real Saudi government hospital environment.					X

Note. Signed evaluation form reproduced with the reviewer's permission; synthetic data only.

Figure D6

Reviewer 2 Evaluation Form, Page 3 (Open Feedback and Declaration)

Section D. Open feedback

D1. Main strengths of the framework (two or three points)
1. User-Centric Clinical Workflow Alignment: The integration of context-aware policies (role, duty status, patient assignment) seamlessly bridges security constraints with fast-paced hospital operations. 2. Robust Emergency Protocols: The break-glass feature is meticulously designed with mandatory justification and automatic 15-minute expiration, solving a critical healthcare security dilemma. 3. Audit Traceability & Compliance: The tamper-evident logging mechanism ensures high accountability, aligning well with regional healthcare regulatory standards (such as SFDA and CBAHI guidelines).

D2. Weaknesses, risks or gaps you noticed
1. User Onboarding and Training Overhead: Clinicians might require initial orientation to understand step-up MFA prompts and context validation rules during hectic shifts. 2. Centralized Directory Dependency: High reliance on real-time directory services for context checks could introduce minor bottlenecks if redundancy isn't fully optimized.

D3. Suggested improvements before a hospital pilot
1. Implement automated caching mechanisms for offline or degraded network scenarios to maintain clinical continuity. 2. Provide role-specific micro-training modules directly within the user interface for first-time users. 3. Expand the audit export feature to support standard SIEM connectors (e.g., Splunk, ELK stack) for hospital SOC integration.

D4. Would you recommend piloting this framework in a hospital setting after further development? Yes / [X] Yes, with the changes above / No

Section E. Declaration

I confirm that this evaluation reflects my independent professional judgement, that I reviewed the prototype described in Section B, and that no real patient data was shown to me.

Reviewer signature	Date
Abdulrahim Yousif Almirabi 	October 24, 2026

Please return the completed form to the student by email. Thank you for your time and expertise.

Note. Signed evaluation form reproduced with the reviewer's permission; synthetic data only.

Appendix E: Code Excerpts

Service orchestration (src/ehr_access_service.py), reproduced from the released commit:

```
def process_request(self, request: AccessRequest) -> DecisionResult:
    """Evaluate one request and store its audit event."""
    result = self.engine.evaluate(request)
    event = self._find_audit_event(result.audit_event_id)
    self.repository.save_event(event)
    return result
```

Audit signing and verification (src/access_control_engine.py):

```
def _sign(self, payload: dict[str, object]) -> str:
    encoded = json.dumps(
        payload, sort_keys=True, separators=(",", ":")
    ).encode()
    return hmac.new(
        self.audit_secret, encoded, hashlib.sha256
    ).hexdigest()

def verify_audit_event(self, event: AuditEvent) -> bool:
    expected = self._sign({...event fields except checksum...})
    return hmac.compare_digest(event.checksum, expected)
```